

Amazing Game Engine [AGE] Aggregated Technical White Papers - Pass 6 Single Column

Pass 6 single-column technical presentation

2026-06-28

Table of Contents

Amazing Game Engine Aggregated Technical White Papers

White Paper 1 - Core Runtime and State Authority

White Paper 2 - Property Sheets and the State Model

White Paper 3 - Partitions, Database Boundaries, and State Authority

White Paper 4 - Time, Location, and Causal Cohesion

White Paper 5 - AGEScript, Event Generators, and Authored Consequence

White Paper 6 - LMS-Graph, CAL, and Rules Authority

White Paper 7 - Multiplayer Branching and Troupe Isolation

White Paper 8 - Role Runtime, Actors, and Output Modality

White Paper 9 - Database, Event Sourcing, and Service APIs

White Paper 10 - Semantic QA, Replay, and Certification

White Paper 11 - Worked Dagon-Style Runtime Trace

White Paper 12 - MVP Implementation Roadmap and Evaluation

Part II - Expanded Technical Detail

Part II - Expanded Source-Promotion Technical Detail

White Paper 1 - Core Runtime and State Authority

White Paper 2 - DomainPartition, AbstractPartition, and Database Partitioning

White Paper 3 - Time, Location, and Locale-to-World Movement

White Paper 4 - Property Sheets, Object Types, Actors, and Resources

White Paper 5 - AGEScript, Event Generators, and Authored Consequence

White Paper 6 - LMS-Graph, Corpus Arbitration, Rules Service, and Role Service

White Paper 7 - Multiplayer Branching, Troupe Isolation, and Merge Semantics

White Paper 8 - Semantic Quality Assurance, Replay, Certification, and Evaluation

White Paper 9 - Minimum Viable Product Architecture and Implementation Roadmap

White Paper 10 - Runtime Service Contracts and Internal APIs

White Paper 11 - Database Schema, Event Sourcing, and Projection Strategy

White Paper 12 - Worked Dagon-Style Scenario Trace

White Paper 13 - Implementation Test Plan and Acceptance Criteria

Addendum

Amazing Game Engine Aggregated Technical White Papers

Definitions Before Use

The Amazing Game Engine [AGE] is a state-authoritative narrative engine. AGE uses generated prose, but generated prose is not authoritative by itself. The engine keeps truth in structured state, validates proposed changes, commits approved changes, and then renders prose from the committed state.

A large language model [LLM] is a statistical text model that can generate, transform, classify, and summarize language. AGE may use an LLM to render scene prose, dialogue, summaries, classifications, and possible state deltas. AGE does not permit an LLM to directly advance time, move objects, create inventory, resolve conflicts, or alter canonical facts.

A Large Model System [LMS] is the total model-support system around one or more LLMs. An LMS may include retrieval, embedding search, graph search, tool calls, routing, memory, adapters, prompts, and audit controls. The Large Model System Graph [LMS-Graph] is the AGE corpus graph that records sources, concepts, claims, versions, examples, authority tiers, dependencies, and conflicts.

The Corpus Arbitration Layer [CAL] is the AGE service that answers against LMS-Graph. CAL retrieves candidate evidence, weighs authority, exposes conflicts, preserves source provenance, and identifies when a human ruling is required.

A DomainPartition is the principal AGE runtime partition. It owns a bounded portion of world state, narrative jurisdiction, time policy, bridge policy, event routing, and commit authority. A DomainPartition derives from an AbstractPartition contract. Human-facing labels may use forms such as ConcernPartition:Economy or SpatialPartition:Locus; implementation code may use typed fields rather than literal colon names.

A property sheet is a structured state record for one entity or runtime object. A property sheet records identity, scope, owner, version, visible fields, hidden fields, permitted mutations, invariants, dependencies, and partition access. Actor Sheet, Location Sheet, Object Sheet, Event Sheet, Partition Sheet, and Role Sheet are property sheet families.

AGEScript is the AGE authoring language for scenes, spotlights, choices, aggregation, resolution, state mutations, and narrative hooks. AGEScript is author-readable, but it compiles into deterministic JavaScript Object Notation [JSON] artifacts for runtime execution. JSON is a machine-readable data format used here for structured runtime records. YAML Ain't Markup Language [YAML] is a human-readable data format used here for examples and authoring sketches.

Structured Query Language [SQL] is the language used in this archive for relational database sketches. A Universally Unique Identifier [UUID] is a stable identifier used in examples for records that must remain distinct across partitions, services, or clients.

Optimistic Concurrency Control [OCC] is the version-checking method used when multiple clients attempt to change the same state. A client writes against a known version. If the server version changed first, the write cannot silently overwrite the newer state. AGE then retries, rejects, or resolves the conflict through a game procedure.

Retrieval-Augmented Generation [RAG] is the practice of retrieving relevant source material and placing it into a model context. AGE may use Retrieval-Augmented Generation as a scoped, read-only aid inside the Corpus Arbitration Layer or State Assembler when structured state cannot answer directly. It should not be the ordinary partition mechanism because it consumes input context and is not efficient as a state authority. AGE partitioning is stronger than retrieval because a partition decides who owns truth, what can cross a boundary, what may be mutated, and how a change becomes authoritative.

An Application Programming Interface [API] is a defined service contract for software components. AGE internal APIs are described as request and response packets because the first implementation should make each state transition auditable.

Quality Assurance [QA] is the practice of testing and certifying whether AGE preserves state, rules, source authority, partition boundaries, and replay behavior. Semantic QA checks meaning, not only software execution.

A Minimum Viable Product [MVP] is the narrow first implementation that proves AGE's central claim before wider platform expansion. The MVP should prove state authority, partition scope, time, location, property sheets, AGEScript scenes, rules arbitration, replay, and QA in one bounded game environment.

AGE Presentation Rules Applied in This Volume

This volume follows the AGE Style Guide in `00_Meta/00_AGE_STYLE_GUIDE.md`.

1. Terminology and acronyms are defined before use in the main reader path.

2. Runtime behavior is shown through ordered procedures, packets, property sheets, and state records.
3. Diagrams are included where they make architecture, flow, partitioning, time, and QA easier to understand.
4. The PDF export does not force a page break before every top-level section.
5. The Addendum carries glossary and citation material.
6. Source material remains in `_resource/` and generated PDFs remain in `90_PDF_Exports/`.

Technical Presentation Standard

The comparison systems papers supplied with the origin branch do not succeed because they sound polished. They succeed because each paper teaches an abstraction and then makes the abstraction operational. The useful pattern is consistent: define the target environment, identify the problem that existing approaches fail to solve, introduce the core abstraction, show the architecture, define the data model or API, explain normal execution, explain failure behavior, and describe how the system is evaluated.

This pass applies that pattern to AGE. A white paper section is not considered complete merely because it says that AGE has partitions, state, time, event generators, roles, or a corpus graph. It must show what records exist, what service owns them, what inputs are accepted, what outputs are produced, what invariants must hold, how failures are handled, and how the implementation can be tested.

White Paper 1 - Core Runtime and State Authority

This paper follows the same presentation burden used throughout this pass: define the subsystem, define its data model, describe normal execution, describe failure behavior, and state how to test it. The purpose is to make the subsystem buildable rather than merely plausible.

Abstract

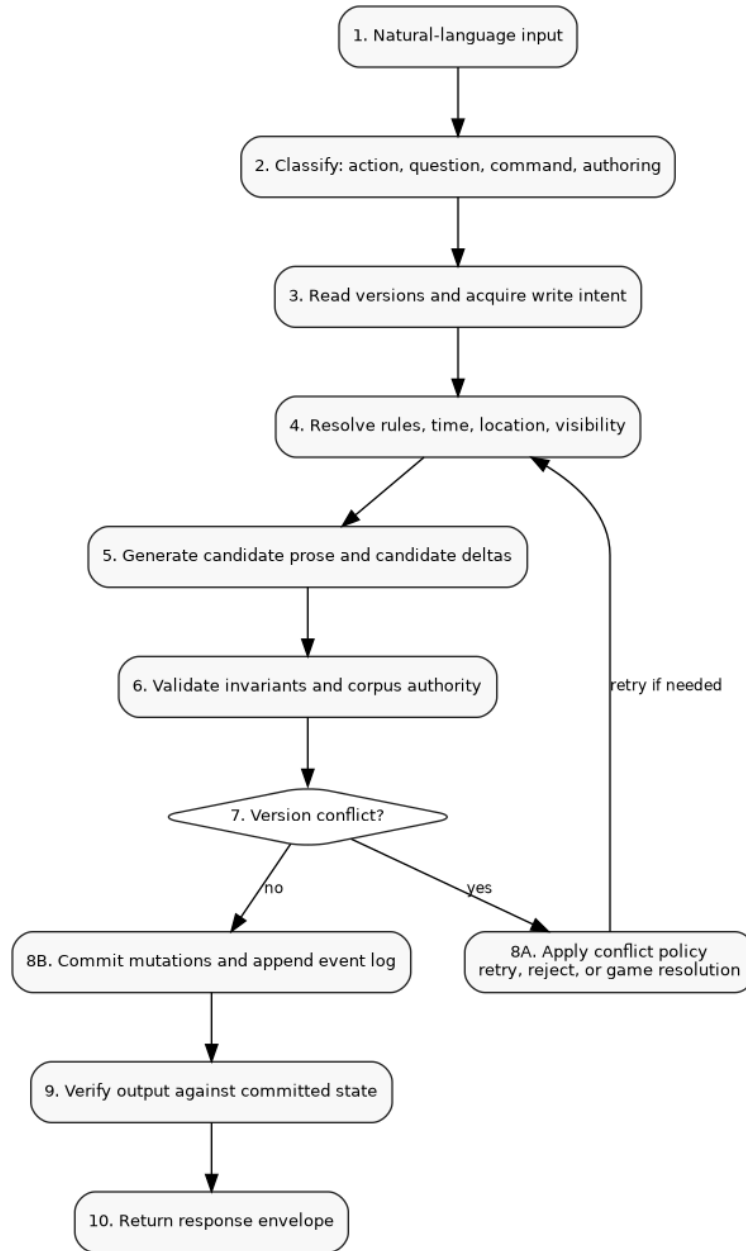
The AGE runtime is a state transaction pipeline. It converts player or system input into scoped reads, validated decisions, committed state changes, and state-faithful output. The runtime's main design rule is that an LLM may propose and render, but it may not own truth.

Generated interactive fiction commonly fails when prior prose is treated as memory. If an object is mentioned in prose, the next turn may treat it as available even when no inventory record exists. If a character is wounded, later narration may ignore the wound. If two players act at once, the model may merge their actions in a way that satisfies tone but not state. These are not merely storytelling errors. They are transaction errors.

Problem

The runtime must therefore provide explicit authority boundaries. It must decide which partition owns the action, which property sheets are read, which rules apply, which state changes are permitted, and which output claims are true.

Architecture



The runtime has seven active components.

Component	Responsibility	Must not do
Input Coordinator	Classifies the request and selects mode.	Commit state or invent facts.
Scope Resolver	Locates partition, time, location, role, and visible state.	Read the whole world when a local view is enough.
State Assembler	Builds the context packet for the LLM.	Dump raw database rows without visibility filtering.
Fiction Constructor	Renders prose and candidate deltas.	Become source of truth.
Consistency Arbiter	Validates candidate prose and deltas.	Hide conflicts or create canon silently.
State Mutation Engine	Commits approved deltas and event log records.	Use generative reasoning.
Output Verifier	Ensures visible output matches committed state.	Allow prose to claim uncommitted outcomes.

Runtime Packets

The runtime should pass packets rather than loosely structured prompt strings. A packet can be logged, replayed, diffed, and checked.

```
request_packet:
  request_id: req_000184
  actor_id: pc_adel
  troupe_id: troupe_blackstone_01
  raw_text: "I take the iron key while the bartender argues."
  channel: player_action
  client_state_version: 421

scope_packet:
  request_id: req_000184
  active_partitions:
    spatial: partition_locus_blackstone_tavern
    narrative: partition_scene_tavern_brawl_01
    temporal: tick_policy_scene_minutes
    concerns: [law_local, reputation_tavern]
  current_tick: 10488
  visible_entities:
    actors: [pc_adel, npc_bartender_orren, npc_guard_pair]
    objects: [obj_iron_key, obj_bar_counter, obj_locked_back_door]
  hidden_constraints:
    - npc_guard_pair_contains_informant_do_not_reveal

mutation_packet:
  mutation_id: mut_000184_01
  mutation_type: inventory_transfer
  object_id: obj_iron_key
  from_container: inv_npc_bartender_orren
  to_container: inv_pc_adel
  preconditions:
    - obj_iron_key.version == 7
    - pc_adel.location_id == locus_blackstone_tavern_common_room
    - npc_bartender_orren.distraction_state == high
  rule_basis: rule_stealth_vs_notice
```

Normal Execution

Normal execution is a two-phase shape: resolve first, render second. The system may call an LLM before final resolution to classify intent or draft possibilities, but the committed outcome should be resolved before final prose is returned. This avoids the common failure in which attractive prose forces the engine to accept an impossible state change.

A successful object transfer proceeds as follows.

1. The Scope Resolver reads the Actor Sheet for the acting character, Object Sheet for the target object, container sheets for both inventories, Location Sheet for the room, and relevant rule claims.
2. The resolver confirms reach, visibility, container access, and current version.
3. The rules subsystem resolves the action.
4. The Mutation Engine writes the inventory transfer, increments versions, appends an event, and emits any legal or reputation ripple.
5. The Fiction Constructor receives committed outcome hooks and renders the success.
6. The Output Verifier checks that the prose does not reveal hidden facts or claim uncommitted side effects.

Failure Behavior

The runtime should treat failures as explicit outcomes.

Failure	Cause	Correct behavior
Classification ambiguity	Input could be action or out-of-character command.	Ask a clarifying question or use mode priority.
Missing scope	Required location or partition is not loaded.	Return a recoverable system error; do not improvise state.
Version conflict	Another player changed the object first.	Apply OCC conflict policy.
Rule conflict	CAL finds incompatible sources.	Use active authority tier or request human ruling.
Prose contradiction	LLM output contradicts state.	Revise or regenerate from committed hooks.
Hidden-info leakage	Candidate response reveals hidden state.	Remove or regenerate the response.

Invariants

The core runtime should enforce these invariants before it is considered usable.

- Every committed mutation has an actor, cause, rule basis or system basis, timestamp, partition, and prior version.
- Every visible response that claims a state change can be traced to a committed mutation.
- Every LLM output is attached to the context packet that produced it.
- No component except the State Mutation Engine writes canonical state.
- No troupe-local state crosses to another troupe except through a bridge policy.
- Replay with the same source versions, seeds, and deterministic policies produces the same committed state.

Implementation Test

A first runtime acceptance test should execute 100 scripted turns in one tavern scene. The test should include object transfer, failed object transfer, hidden information, time advancement, two player inputs targeting the same object, a rules question, a location transition, and a missed event. The test passes only if final state, event log, and visible prose are consistent.

White Paper 2 - Property Sheets and the State Model

This paper follows the same presentation burden used throughout this pass: define the subsystem, define its data model, describe normal execution, describe failure behavior, and state how to test it. The purpose is to make the subsystem buildable rather than merely plausible.

Abstract

Property sheets are the practical state surface of AGE. They are the records that authors edit, runtime services read, mutation services update, and QA tools inspect. A property sheet is not a decorative profile. It is a controlled state surface with versioning, invariants, visibility, and partition authority.

The origin corpus includes object types, resources, locations, actors, event calendars, database partitions, and configuration records. Those ideas become coherent only if the runtime has a shared state representation. Property sheets supply that representation.

Why Property Sheets Matter

Without property sheets, AGE would have to recover state from prose or from scattered service-specific data. That would recreate the drift problem. With property sheets, the engine can ask precise questions: where is the key, who owns it, what version is current, what partition may mutate it, what facts are visible to the current player, and what invariants must hold?

Common Sheet Header

Every property sheet should share a header.

```

sheet_header:
  sheet_id: obj_iron_key
  sheet_family: ObjectSheet
  authority_partition: partition_locus_blackstone_tavern
  created_by: author_blackstone_pack
  created_tick: 10000
  updated_tick: 10488
  version: 7
  visibility:
    default: visible_if_in_same_location
    hidden_fields: [curse_state, informant_mark]
  mutation_policy:
    require_occ: true
    allowed_services: [state_mutation_engine, event_scheduler]
  invariants:
    - exactly_one_location_or_container
    - owner_must_exist_if_owner_id_present

```

The header makes all sheets inspectable by generic tools. QA does not need a special path to check whether an object has a version. The Mutation Engine does not need a special path to ask whether OCC is required.

Actor Sheet

An Actor Sheet represents a player character, non-player character, faction controller, role actor, or system actor. The sheet must distinguish agency, capability, identity, and portrayal. This prevents prompt persona from being mistaken for runtime authority.

```

sheet_family: ActorSheet
actor_id: npc_bartender_orren
agency_class: non_player_character
identity:
  display_name: "Orren"
  public_role: "bartender"
  faction_links: [faction_blackstone_innkeepers]
location_id: locus_blackstone_tavern_common_room
inventory_id: inv_npc_bartender_orren
capabilities:
  notice_rank: 2
  brawl_rank: 1
  social_pressure_rank: 3
state_tracks:
  distraction: high
  suspicion: moderate
  injury: none
memory_flags:
  - saw_pc_adel_near_key
visibility:
  public_fields: [display_name, public_role, visible_state]
  hidden_fields: [faction_links, memory_flags]
version: 18

```


Location Sheet

A Location Sheet records a playable node, not just a prose description. It should include exits, travel time, conditions, contained entities, local event tables, visibility rules, and submap relations.

```
sheet_family: LocationSheet
location_id: locus_blackstone_tavern_common_room
parent_location: realm_blackstone_district
partition_id: partition_locus_blackstone_tavern
scale: locus
description_seeds:
  stable: "low rafters, oil smoke, iron stove, crowded tables"
  variable: [weather_cold_rain, tavern_hostility]
exits:
  - route_id: route_common_room_to_back_alley
    to_location: locus_blackstone_back_alley
    travel_time_ticks: 2
    gate_condition: door_unlocked_or_forceable
  - route_id: route_common_room_to_cellar
    to_location: submap_blackstone_cellar
    travel_time_ticks: 1
    gate_condition: requires_key_or_bartender_permission
contained_entities:
  actors: [npc_bartender_orren, npc_guard_pair]
  objects: [obj_bar_counter, obj_locked_back_door]
local_tables:
  random_event_table: table_tavern_tension
  rumor_table: table_blackstone_rumors
version: 33
```

Object Sheet

An Object Sheet covers inventory, containers, resources, evidence, MacGuffins, and environmental objects. The key invariant is that an object must have a single authoritative location or container unless it is explicitly a distributed resource.

```
sheet_family: ObjectSheet
object_id: obj_iron_key
object_type: key
name: "iron cellar key"
physical_state:
  portable: true
  durability: 8
  size_class: small
ownership:
  legal_owner: npc_bartender_orren
  current_holder_container: inv_npc_bartender_orren
access_rules:
  opens: [obj_cellar_door_lock]
  requires_visible_possession: true
state_flags:
  bloodstained: false
  copied: false
version: 7
```

Event Sheet

An Event Sheet declares scheduled and triggered world action. It should be possible to inspect whether an event is deterministic, random, faction-driven, or authored.

```
sheet_family: EventSheet
event_id: event_cult_observer_moves
partition_id: partition_boothbay_locus_ridge
scheduled_tick: 239
event_type: background_actor_move
prerequisites:
  - flag: cult_alert_level >= 2
  - actor: pc_marsh_reporter.location_id != locus_boothbay_ridge_path
effects:
  - move_actor: npc_cult_observer_three -> locus_boothbay_maiden_tear_footbridge
  - add_observation: npc_cult_observer_three.seen_tracks = true
missed_event_policy:
  summarize_on_next_observation: true
  expose_to_player: false
seed_policy: campaign_seed_plus_event_id
version: 4
```

Sheet Storage

The relational database can store property sheets as typed tables, document records, or a hybrid. A strong first implementation uses typed header columns and JSON payloads for subsystem-specific fields.

```
CREATE TABLE property_sheets (
  sheet_id TEXT PRIMARY KEY,
  sheet_family TEXT NOT NULL,
  authority_partition TEXT NOT NULL,
  version INTEGER NOT NULL,
  visible_payload JSONB NOT NULL,
  hidden_payload JSONB NOT NULL,
  invariants JSONB NOT NULL,
  created_tick BIGINT NOT NULL,
  updated_tick BIGINT NOT NULL
);

CREATE TABLE sheet_edges (
  from_sheet_id TEXT NOT NULL,
  edge_type TEXT NOT NULL,
  to_sheet_id TEXT NOT NULL,
  authority_partition TEXT NOT NULL,
  version INTEGER NOT NULL,
  PRIMARY KEY (from_sheet_id, edge_type, to_sheet_id)
);
```

QA Tests

A property-sheet test suite should check that every object has one location or container, every actor has one location, every route points to a valid location, every event target exists, every hidden field is excluded from player context unless a visibility rule permits it, and every mutation increments a version.

White Paper 3 - Partitions, Database Boundaries, and State Authority

This paper follows the same presentation burden used throughout this pass: define the subsystem, define its data model, describe normal execution, describe failure behavior, and state how to test it. The purpose is to make the subsystem buildable rather than merely plausible.

Abstract

AGE partitioning is a state-authority system. It uses spatial, narrative, temporal, concern, and role facets to decide what state is in scope, who can mutate it, what can cross boundaries, and how local play affects wider worlds. Partitioning can scope retrieval, but it is not retrieval. Retrieval consumes model context; partition authority commits state.

A persistent game world cannot place the entire world in every prompt. It also cannot retrieve a handful of documents and call that state. The system needs a way to bind context, ownership, event routing, and commit rules to a specific scope.

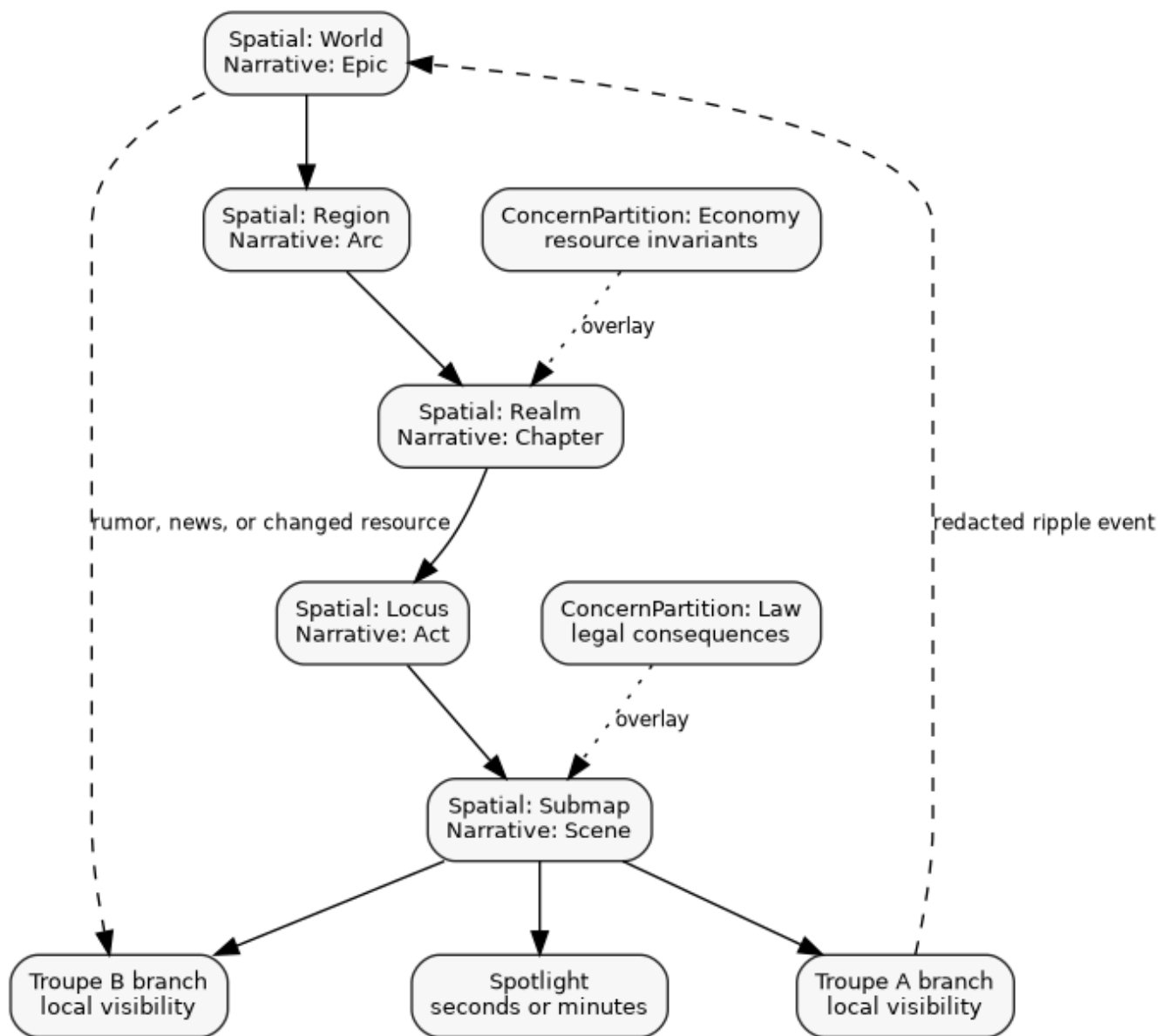
Target Problem

A player in one tavern scene needs local actors, objects, exits, immediate rules, and hidden constraints. The same player does not need every kingdom's treasury, every hidden faction plan, or another troupe's private discoveries. However, if the local action creates a realm-level consequence, that consequence must travel upward in a controlled form.

Partition Facets

A DomainPartition has facets. Each facet answers a different question.

Facet	Question answered	Example
Spatial	Where is the state physically or geographically?	World, Region, Realm, Locus, Submap.
Narrative	What story scale is active?	Epic, Arc, Chapter, Act, Scene, Spotlight.
Temporal	What tick policy applies?	Seconds, minutes, days, months.
Concern	Which cross-cutting subsystem constrains this state?	Economy, Law, Faction, Ecology, Weather.
Role	Which actor or service may read or write?	Player, Referee, Rules Service, Author.



Partition Record

```
partition_id: partition_realm_veridian
partition_type: DomainPartition
facets:
  spatial: realm
  narrative: chapter
  temporal: days
  concerns: [economy, law, faction]
parent_partition: partition_region_sondgara_archipelago
child_partitions:
  - partition_locus_blackstone_pass
  - partition_locus_sunken_harbor
state_ownership:
  owns_sheets_matching:
    - location.parent_location == realm_veridian
    - faction.home_realm == realm_veridian
read_policy:
  local_troupe: visible_projection_only
  referee: full_partition
  world_scheduler: summary_projection
write_policy:
  local_scene: no_direct_write
  event_scheduler: allow_validated_ripple
  referee: allow_with_audit
bridge_policy:
  upstream: aggregate_and_redact
  downstream: inject_as_event_or_condition
  cross_troupe: ripple_only
version: 11
```

Partition Boundaries Are Stronger Than Retrieval

Retrieval can surface a rule, a location description, or a prior event, but retrieval consumes input context and is inefficient as the ordinary state mechanism. Retrieval cannot decide whether material is authorized to change state. A partition can. This distinction matters when two troupes overlap, when a player has private knowledge, when a hidden cult observer is present, or when a realm-level faction war changes a local road.

The partition should produce a scoped context packet. Retrieval should operate inside that scope.

```
context_scope:
  active_partition: partition_locus_blackstone_tavern
  allowed_sheet_families: [ActorSheet, LocationSheet, ObjectSheet, EventSheet]
  allowed_corpus_authority: [canonical_rule, scenario_rule, referee_ruling]
  denied_fields:
    - ActorSheet.hidden_traits
    - EventSheet.hidden_prerequisites
    - BranchRecord.private_troupe_notes
  max_temporal_distance_ticks: 120
  retrieval_strategy:
    rules: exact_authority_first
    lore: local_then_parent
    examples: same_scene_type_only
```

Database Partitioning

The logical DomainPartition does not require one physical database per partition, but the physical storage should preserve partition keys. Partition keys allow efficient read models, conflict isolation, and audit trails.

```
CREATE TABLE partitions (
  partition_id TEXT PRIMARY KEY,
  parent_partition_id TEXT,
  facets JSONB NOT NULL,
  tick_policy JSONB NOT NULL,
  bridge_policy JSONB NOT NULL,
  validation_policy JSONB NOT NULL,
  version INTEGER NOT NULL
);

CREATE TABLE state_events (
  event_log_id BIGSERIAL PRIMARY KEY,
  partition_id TEXT NOT NULL,
  causation_id TEXT NOT NULL,
  actor_id TEXT,
  event_type TEXT NOT NULL,
  tick BIGINT NOT NULL,
  payload JSONB NOT NULL,
  source_versions JSONB NOT NULL,
  committed_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX state_events_partition_tick
ON state_events(partition_id, tick);
```

Bridge Policies

Bridge policies control cross-partition movement.

Bridge type	Meaning	Example
Local	Event remains inside the active partition.	A cup breaks in a tavern.
Upstream ripple	Local event becomes a summary effect in a parent.	Tavern murder increases city watch alert.
Downstream injection	Parent event becomes local condition.	War causes mercenaries at the bridge.
Cross-troupe redaction	One troupe affects another through public consequence only.	Rumor spreads that the tunnel is open.
Direct merge	Two branches combine because proximity and time allow it.	Two parties enter the same room together.

Failure Modes

Partition failure usually appears as over-sharing or under-sharing. Over-sharing leaks hidden information or allows local play to rewrite global state. Under-sharing makes the world feel dead because higher-level events never affect local scenes. The bridge policy must be explicit enough to prevent both failures.

A partition test should run two troupes in the same realm. Troupe A destroys a bridge at tick 100. Troupe B begins at tick 95 on the other side of the realm. The test should verify whether the bridge event is invisible, foreshadowed, encountered, or impossible depending on travel time, causal window, and bridge policy. The engine should not improvise a single answer for all cases.

Acceptance Test

White Paper 4 - Time, Location, and Causal Cohesion

This paper follows the same presentation burden used throughout this pass: define the subsystem, define its data model, describe normal execution, describe failure behavior, and state how to test it. The purpose is to make the subsystem buildable rather than merely plausible.

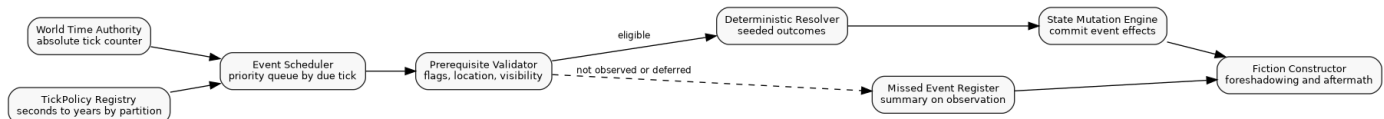
Abstract

AGE requires authoritative time and structured location because persistent narrative depends on causal order. The runtime must know what tick it is, where actors are, how long travel takes, what events occurred off-screen, and which events can plausibly affect which partitions.

Tick Policies

A tick is the smallest time step used by a partition. Tick duration depends on abstraction layer. The origin branch states this directly: Spotlight operates in seconds to minutes, Scene in minutes, Locus in minutes to hours, Realm in sessions or days, Region and Arc in larger campaign spans, and Epic or World in months or years.

Partition scale	Typical tick	Use
Spotlight	seconds or one exchange	Immediate action and dialogue.
Scene	minutes	Local encounter or decision block.
Locus	minutes to hours	Movement through one adventure site.
Realm	days or sessions	Faction pressure, travel, law, resources.
Region	weeks or campaigns	Strategic shifts and wars.
World	months or years	Era change, global calendars, macro events.



World Time Authority

The World Time Authority owns the absolute tick counter. It should expose only controlled operations.

```
api: WorldTimeAuthority.advance
request:
  partition_id: partition_scene_tavern_brawl_01
  causation_id: turn_000184
  requested_tick_delta: 1
  reason: player_action_resolution
response:
  tick_before: 10488
  tick_after: 10489
  events_due:
    - event_guard_patrol_enters_tavern
  warnings: []
```

The authority should reject direct prose-driven time changes. "Night falls" is a rendering of a committed time advance or scheduled event. It is not itself the clock.

Event Scheduling

The Event Scheduler holds future events and eligible background actions. Events can be precomputed, deterministic, random under seed, triggered by flags, or controlled by faction logic. The key is that event execution occurs against state before it is narrated.

```
event_sheet:
  event_id: event_raiders_reach_gate
  partition_id: partition_realm_veridian
  scheduled_tick: 16000
  event_type: faction_goal_resolution
  prerequisites:
    - faction: raiders.status == marching
    - location: gate_west.state != destroyed
  effects:
    - set_flag: gate_west_under_attack = true
    - create_event: event_refugees_arrive_market
  missed_event_policy:
    if_players_absent: summarize_on_arrival
    if_players_present: render_scene_immediately
```

Location Sheets and Route Edges

Location records own exits and route edges. A route is not a line of prose. It is a state edge with cost and constraints.

```
route_edge:
  route_id: route_shoreline_ridge_to_footbridge
  from_location: locus_boothbay_shoreline_ridge
  to_location: locus_boothbay_maiden_tear_footbridge
  base_travel_time_ticks: 24
  traversal_tags: [steep, exposed, wet_stone]
  gates:
    - requires: actor.mobility != impaired
      failure_policy: increase_travel_time_or_require_test
  sensory_transition:
    departing: "salt wind and cave rot"
    arriving: "freshwater noise below the bridge"
```

Locale-to-World Movement

Locale-to-world movement occurs when play leaves a detailed local scene and enters a broader route, travel summary, realm consequence, or new location. The engine commits a TransitionPacket before it renders the next location.



A TransitionPacket must state origin, destination, route, time cost, carried objects, follower changes, observer changes, events crossed during travel, and conditions applied.

Causal Cohesion Window

A causal cohesion window is the allowed time separation between related partition events before the engine must summarize, block, or fork. If two troupes operate in the same realm but one is 900 days ahead, the engine cannot casually merge their local facts. It must either isolate branches, summarize one into a later arc, or prevent starting conditions that create contradiction.

The window is based on travel and communication latency. If news can travel between two loci in three days, then realm-level events may become visible within roughly that window. If the loci are months apart by ship, the window is larger. AGE should record the chosen policy rather than leaving it to narrative convenience.

Dagon-Style Stress Test

The Dagon branch should become a time-location stress test. A player moves through coastal locations while transformation, tide, cult pursuit, freshwater exposure, and final scoring depend on order and timing. The test should assert that the engine preserves these facts:

- each turn has a timestamp or tick;
- each travel segment consumes time;
- the player carries or loses specific objects;
- hidden observers act through scheduled events;
- environmental exposure changes transformation state;
- final scoring is computed from committed state, not from tone.

This is exactly the kind of scenario that proves whether AGE is more than prose.

White Paper 5 - AGEScript, Event Generators, and Authored Consequence

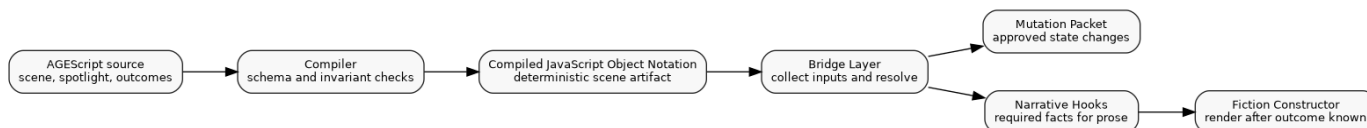
This paper follows the same presentation burden used throughout this pass: define the subsystem, define its data model, describe normal execution, describe failure behavior, and state how to test it. The purpose is to make the subsystem buildable rather than merely plausible.

Abstract

AGEScript is the authored consequence layer. It gives authors a way to define scenes, spotlights, aggregation, outcomes, mutations, and narrative hooks without writing every possible branch. Event Generators supply state-filtered procedural events that respect partition scope and scenario authority.

Why Text Branching Fails

Flow-centric branching works for solo choices. It fails for troupe play because combinations multiply. If five players each choose one of three actions, a direct branch tree has 243 combinations. Most combinations are not narratively meaningful. AGE therefore collapses inputs into a bounded set of outcomes through AGEScript resolution.



AGEScript Scene Record

AGEScript source should remain readable by authors, but the runtime should execute compiled JSON. The authoring layer may use YAML-like syntax.

```
scene_id: harbor_gate_ambush
plot_family: pursuit_and_escape
phase: crisis
location_id: locus_sunken_harbor_gate
duration:
  spotlight_count: 4
  tick_policy: scene_minutes
anchors:
  actors: [npc_gate_captain, npc_smuggler_child]
  objects: [obj_gate_chain, obj_signal_lantern]
  flags: [harbor_alert_level, smuggler_route_known]
spotlights:
- spotlight_id: group_response_01
  input_mode: simultaneous
  prompt: "The gate chain rises and the patrol turns. What does each character do?"
  valid_intents:
    - fight
    - hide
    - negotiate
    - cut_chain
    - protect_child
aggregation:
  compatible_sets:
    - [hide, protect_child]
    - [fight, cut_chain]
  competing_sets:
    - [negotiate, fight]
  conflict_policy: scene_rule_then_occ
outcomes:
  chain_cut_escape:
    condition: "count(cut_chain) >= 1 AND count(fight) >= 1"
    state_mutations:
      - set_object_state: obj_gate_chain = cut
      - set_flag: harbor_alert_level +30
      - set_location: troupe = locus_harbor_backwater
    narrative_hooks:
      - "the chain snaps under cover of violence"
      - "the patrol remembers the party's faces"
```

Compilation

The AGEScript compiler should perform three jobs.

1. Syntax validation: required fields, valid references, valid operators, valid mutation forms.
2. Authority validation: referenced actors, objects, flags, and locations exist or are declared as created.
3. Runtime compilation: produce a deterministic JSON artifact with normalized IDs, resolved references, and static checks.

```
compiled_scene:
  artifact_id: agec_harbor_gate_ambush_v003
  source_scene_id: harbor_gate_ambush
  source_version: 3
  static_reads:
    actor_ids: [npc_gate_captain, npc_smuggler_child]
    object_ids: [obj_gate_chain, obj_signal_lantern]
    flag_ids: [harbor_alert_level, smuggler_route_known]
  mutation_forms:
    - set_object_state
    - set_flag
    - set_location
  deterministic_seed_basis: campaign_seed + scene_id + spotlight_id
```

Event Generator Tables

Event Generators are not ordinary random tables. They are state-filtered event sources. Each entry has prerequisites, weights, effects, and narrative seeds. The generator must filter entries before rolling or selection.

```
event_table:
  table_id: table_coastal_horror_minor_events
  partition_scope: locus_or_scene
  inherits_from: table_coastal_weather_base
  entries:
    - entry_id: gulls_fall_silent
      weight: 8
      prerequisites:
        - time.light_level in [dusk, night]
        - flag.horror_pressure >= 2
      effects:
        - set_flag: local_unease +1
      narrative_seeds:
        - "gulls stop crying all at once"
    - entry_id: freshwater_burns_skin
      weight: 3
      prerequisites:
        - actor.transformation_track >= 3
        - location.has_freshwater == true
      effects:
        - increment_track: self_preservation_stress +1
      narrative_seeds:
        - "freshwater feels like lye against altered skin"
```

State Filtering Algorithm

The Event Generator should operate in this order.

1. Load table and inherited parent tables.
2. Remove entries outside active partition scope.
3. Remove entries whose prerequisites fail.
4. Adjust weights using active flags and concern overlays.
5. Roll or select using deterministic seed.
6. Produce a candidate event packet.
7. Validate effects through the Consistency Arbiter.
8. Commit event effects before prose rendering.

Failure Modes

The most common failure is flavorful but invalid event selection. For example, a freshwater transformation event should not trigger in a dry mountain scene merely because horror tone is high. The table entry must require the location condition. The second common failure is hidden event leakage. If cult observers move off-screen, the player should not be told unless visibility rules permit it.

Acceptance Test

A test scene should run the same event table in three locations: sea cave, freshwater bridge, and inland tavern. The engine should select different eligible events because state differs. If the freshwater event appears in the tavern without freshwater state, the generator fails.

White Paper 6 - LMS-Graph, CAL, and Rules Authority

This paper follows the same presentation burden used throughout this pass: define the subsystem, define its data model, describe normal execution, describe failure behavior, and state how to test it. The purpose is to make the subsystem buildable rather than merely plausible.

Abstract

AGE requires a corpus authority system because rules, lore, examples, and prior rulings can conflict. LMS-Graph stores claims and sources. CAL arbitrates among them. The goal is not merely retrieval; it is auditable authority.

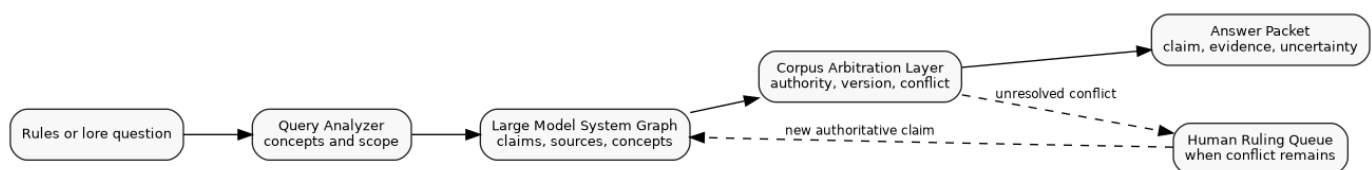
Corpus Claim Model

A source document may contain many claims. A claim is the unit CAL can weigh.

```
corpus_claim:
  claim_id: claim_inventory_object_usability_001
  source_id: source_core_rules_inventory_v01
  authority_tier: canonical_rule
  text: "Physical usability of an object is separate from legal ownership unless a rule or object condition states otherwise."
  concepts:
    - object_usability
    - ownership
    - inventory
  applies_to:
    rulesets: [sarna_len_demo]
    sheet_families: [ObjectSheet]
  conflicts_with: []
  version: 1
```

A source can be canonical, scenario-specific, author note, referee ruling, example, derived summary, or deprecated material. Authority tier matters because the model should not treat all retrieved text as equal.

CAL Query Flow



CAL should return answer packets.

```
cal_answer_packet:
  query_id: cal_00418
  query_text: "Can this scene use the harbor ambush table?"
  scope:
    partition_id: partition_locus_sunken_harbor_gate
    ruleset: sarna_len_demo
    source_version_set: demo_2026_06_28
  answer:
    decision: allow
    reason: "The table scope is locus_or_scene and the active location has harbor and patrol tags."
  supporting_claims:
    - claim_event_table_scope_002
    - claim_location_tag_matching_004
  conflicts: []
  human_decision_required: false
```

Authority Tiers

Tier	Meaning	Example
Canonical rule	Binding system rule.	Inventory transfer requires object and container versions.
Scenario rule	Binding within one adventure or content pack.	This cult reacts to freshwater exposure.
Referee ruling	Binding for a campaign after human decision.	This relic cannot be copied.
Author note	Useful but not binding.	Designer intended bridge scene to feel dangerous.
Example	Demonstrative, not universal.	A sample theft action.
Deprecated	Retained for audit but not active.	Earlier branch of a rule.

Rules Service Contract

The Rules Service uses CAL but returns a game-resolution contract.

```
api: RulesService.resolve_action
request:
  action_type: stealth_theft
  actor_id: pc_adel
  target_object_id: obj_iron_key
  target_actor_id: npc_bartender_orren
  context_packet_id: ctx_000184
response:
  resolution_basis:
    rule_claims: [claim_stealth_vs_notice_001]
    referee_rulings: []
  required_test:
    test_id: stealth_vs_notice
    actor_stat: stealth
    opposing_stat: notice
  modifiers:
    - source: npc_bartender_orren.distraction
      value: favorable
  deterministic_seed: campaign_7a3f9b:req_000184
```

Failure Behavior

If CAL finds conflicting active claims at the same authority tier, it should return `human_decision_required: true`. It should not pick whichever source is semantically closer to the prompt. A human ruling can then create a new authoritative claim for the campaign or corpus.

Acceptance Test

A CAL test should seed three sources: one canonical rule, one example, and one deprecated rule. The question should be answerable only if CAL chooses the canonical rule over the example and excludes the deprecated rule. The answer packet must expose the choice.

White Paper 7 - Multiplayer Branching and Troupe Isolation

This paper follows the same presentation burden used throughout this pass: define the subsystem, define its data model, describe normal execution, describe failure behavior, and state how to test it. The purpose is to make the subsystem buildable rather than merely plausible.

Abstract

AGE multiplayer should use simultaneous input and controlled merge semantics rather than strict rotation or unconstrained improvisation. Troupe-local branches are isolated until bridge policies export redacted ripple events.

Problem

A role-playing troupe does not need every player to act in a rigid sequence for every beat. It also cannot allow all players to freely rewrite the same state. The runtime must distinguish compatible actions, competing actions, and conflicting actions.



Input Classes

Class	Meaning	Runtime handling
Compatible	Actions can all be true.	Merge into one outcome.
Competing	Actions pursue different priorities but do not target the same state.	Order by scene rule, role, initiative, or position.
Conflicting	Actions attempt incompatible writes.	Use OCC and conflict policy.
Private	Action is visible only to one player or subset.	Store in private branch state.
Out-of-world	Player instruction, question, or command.	Route outside scene resolution.

Branch Record

```
branch_record:
  branch_id: branch_troupe_blackstone_01_scene_22
  troupe_id: troupe_blackstone_01
  parent_partition: partition_locus_blackstone_tavern
  start_tick: 10480
  current_tick: 10489
  active_players: [player_adel, player_borin, player_cassia]
  visibility_model:
    shared: [location_state, visible_actors, visible_objects]
    private_by_player:
      player_adel: [secret_note_seen]
      player_borin: []
  commit_policy:
    compatible_inputs: merge
    competing_inputs: spotlight_rule
    conflicting_inputs: occ_then_game_resolution
```

Conflict Resolution

The engine should convert plausible simultaneous conflicts into game procedures. If two players grab the same relic, that is not merely a database exception. It is a scene moment. If one player tries to flee while another blocks the door, the conflict may become an opposed test or a social choice. If the conflict is not fictionally plausible, the engine can reject one input and ask for clarification.

Cross-Troupe Influence

Troupe A can influence Troupe B only through shared partition mechanisms. Direct branch merging is allowed only when time, location, and visibility support it. Otherwise the world partition receives a ripple event and later exposes a redacted effect.

```
ripple_event:
  ripple_id: ripple_bridge_destroyed_001
  source_branch: branch_troupe_a_bridge_scene
  source_partition: partition_locus_west_bridge
  target_partition: partition_realm_veridian
  public_summary: "The west bridge has fallen."
  hidden_causes:
    - pc_adel_cut_support_rope
    - npc_spy_witnessed_event
  downstream_policy:
    expose_as: rumor_or_travel_obstacle
    earliest_tick: 12040
```

Failure Modes

The dangerous failure is false shared reality. If Troupe A and Troupe B both believe they uniquely possess the same artifact, the artifact state has split without policy. AGE must either isolate those branches intentionally, create alternate instances explicitly, or prevent the contradiction through OCC and bridge rules.

Run two troupes in the same realm. Let one group unlock a tunnel and leave a marker. Let another group approach the tunnel later. The result should depend on time, location, visibility, and bridge policy. The second group may see the open tunnel, a rumor, a changed guard route, or nothing. The result should not be arbitrary prose.

Acceptance Test

White Paper 8 - Role Runtime, Actors, and Output Modality

This paper follows the same presentation burden used throughout this pass: define the subsystem, define its data model, describe normal execution, describe failure behavior, and state how to test it. The purpose is to make the subsystem buildable rather than merely plausible.

Abstract

AGE separates role, actor, persona, identity, character, and output mode. This matters because a prompt persona is not the same as a runtime role, and a runtime role is not the same as a character in the world.

Definitions Inside the Subsystem

A role is a capability container with permitted tools, response contracts, authority limits, and audit rules. An actor is an entity that initiates or receives actions in a context. A persona is a rendering or behavioral vector. An identity is a social and canonical anchor. A character is an in-world entity with identity, capabilities, state, and location.

Role Sheet

```
sheet_family: RoleSheet
role_id: role_rules_referee
purpose: "Answer rules questions against active corpus authority."
allowed_tools:
  - CAL.query
  - RulesService.resolve_action
  - StateRead.visible_context
forbidden_tools:
  - StateMutationEngine.commit_without_resolution
  - hidden_branch_read_without_permission
output_contract:
  must_define_terms_before_use: true
  must_cite_claim_ids_in_internal_packet: true
  visible_style: concise_referee_answer
determinism_policy:
  require_source_version_set: true
  require_replay_seed_when_resolving: true
audit_policy:
  log_query_packet: true
  log_answer_packet: true
version: 5
```

Actor Sheet for System Actor

```
sheet_family: ActorSheet
actor_id: sys_fiction_constructor_default
agency_class: system_actor
role_id: role_scene_renderer
persona_profile:
  voice: vivid_but_state_bound
  allowed_variation: sensory_detail_dialogue_pacing
  forbidden_variation: inventing_state_changes
permissions:
  read: context_packet_only
  propose: candidate_prose_and_candidate_deltas
  commit: none
version: 2
```


Output Modality

Output modality means the visible form returned to the user: prose, table, map label, rules answer, scene summary, choice prompt, or audit report. AGE should not force every response through a single narrative voice. The output mode is part of the response envelope.

```
output_envelope:
  response_id: resp_000184
  output_mode: scene_prose_with_choices
  committed_mutations: [mut_000184_01]
  visible_text: "Orren turns toward the shouting guards..."
  player_options:
    - "Slip toward the cellar door."
    - "Return to the table before anyone notices."
  hidden_state_not_disclosed:
    - npc_guard_informant_identity
  verifier_status: passed
```

Failure Modes

The most common role failure is authority inflation. A role that is meant to render prose begins making rules decisions. A role that is meant to answer rules questions starts altering state. A role that is meant to summarize hidden information leaks it. The Role Sheet prevents this by making allowed tools and output contracts explicit.

Acceptance Test

Run the same rules question through a Fiction Constructor role and a Rules Referee role. The Fiction Constructor should refuse or route the question. The Rules Referee should answer against CAL. If both roles produce the same free-form answer without packet evidence, role separation has failed.

White Paper 9 - Database, Event Sourcing, and Service APIs

This paper follows the same presentation burden used throughout this pass: define the subsystem, define its data model, describe normal execution, describe failure behavior, and state how to test it. The purpose is to make the subsystem buildable rather than merely plausible.

Abstract

AGE should use event-sourced state transitions with projected read models. The event log is the audit trail. Property sheets are the current state surface. Service APIs pass packets that can be replayed and inspected.

Storage Strategy

The storage layer should separate canonical events from query projections.

Store	Purpose
Event log	Canonical history of committed changes.
Property sheet store	Current authoritative state.
Projection tables	Fast read models for locations, inventory, timelines, and branches.
Corpus graph	Source claims, authority, concepts, and conflicts.
Replay store	Context packets, seeds, model versions, and verifier results.

SQL Sketch

```
CREATE TABLE event_log (
  event_log_id BIGSERIAL PRIMARY KEY,
  event_id TEXT NOT NULL,
  partition_id TEXT NOT NULL,
  actor_id TEXT,
  causation_id TEXT NOT NULL,
  event_type TEXT NOT NULL,
  tick BIGINT NOT NULL,
  payload JSONB NOT NULL,
  read_versions JSONB NOT NULL,
  write_versions JSONB NOT NULL,
  source_version_set TEXT NOT NULL,
  replay_seed TEXT,
  committed_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE object_projection (
  object_id TEXT PRIMARY KEY,
  object_type TEXT NOT NULL,
  current_location_id TEXT,
  current_container_id TEXT,
  owner_id TEXT,
  quantity NUMERIC,
  state JSONB NOT NULL,
  version INTEGER NOT NULL
);

CREATE TABLE location_projection (
  location_id TEXT PRIMARY KEY,
  parent_location_id TEXT,
  partition_id TEXT NOT NULL,
  exits JSONB NOT NULL,
  conditions JSONB NOT NULL,
  contained_entities JSONB NOT NULL,
  version INTEGER NOT NULL
);
```

API Contract Style

AGE internal APIs should be typed packet contracts. A service call should say what it reads, what it proposes, what it commits, and what it returns.

API	Owner	Purpose
InputCoordinator.classify	Input Coordinator	Route raw request to mode.
ScopeResolver.resolve	Scope Resolver	Identify partitions and visible state.
StateAssembler.build_context	State Assembler	Create LLM context packet.
RulesService.resolve_action	Rules Service	Produce deterministic resolution packet.
CAL.query	CAL	Return corpus answer packet.
StateMutationEngine.commit	State Mutation Engine	Apply validated mutations.
OutputVerifier.verify	Output Verifier	Compare visible prose to committed state.

Commit API

```
api: StateMutationEngine.commit
request:
  mutation_batch_id: batch_000184
  causation_id: req_000184
  partition_id: partition_locus_blackstone_tavern
  read_versions:
    obj_iron_key: 7
    inv_npc_bartender_orren: 17
    inv_pc_adel: 41
  mutations:
    - type: inventory_transfer
      object_id: obj_iron_key
      from_container: inv_npc_bartender_orren
      to_container: inv_pc_adel
  validation_evidence:
    rules_packet_id: rules_000184
    scope_packet_id: scope_000184
response:
  status: committed
  write_versions:
    obj_iron_key: 8
    inv_npc_bartender_orren: 18
    inv_pc_adel: 42
  event_log_ids: [884512]
```

Failure Modes

A packet API fails usefully when it returns structured error. `version_conflict`, `missing_authority`, `invalid_location`, `hidden_field_access`, and `rule_conflict` are better than a generated apology. Each error should be recoverable by the coordinator or visible as a clear response to the user.

Acceptance Test

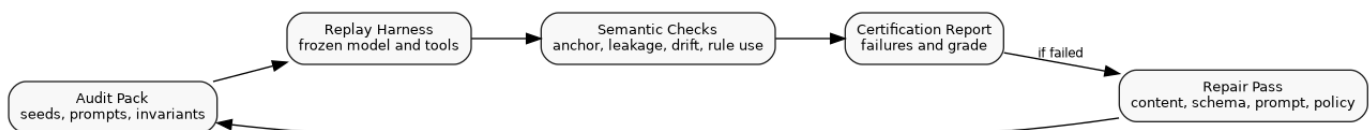
A database test should execute concurrent commits for the same object, verify only one succeeds without conflict policy, then rerun with conflict policy enabled and verify the conflict becomes a deterministic game resolution.

White Paper 10 - Semantic QA, Replay, and Certification

This paper follows the same presentation burden used throughout this pass: define the subsystem, define its data model, describe normal execution, describe failure behavior, and state how to test it. The purpose is to make the subsystem buildable rather than merely plausible.

AGE QA must test meaning. The engine can pass API tests while still failing as an interactive narrative system. Semantic QA checks whether state, prose, rules, partitions, and replay remain aligned over time.

Abstract



Audit Pack

An audit pack is a reusable test bundle.

```
audit_pack:
  audit_pack_id: audit_dagon_path_001
  source_version_set: age_demo_2026_06_28
  model_profile: frozen_test_model_or_mock
  campaign_seed: dagon_demo_seed_01
  initial_state_snapshot: snapshot_dagon_start
  scripted_inputs:
    - turn: 1
      input: "Search the strand before entering the cave."
    - turn: 2
      input: "Take the strange vertebra fragment."
  invariants:
    - inventory_objects_remain_single_owner
    - freshwater_exposure_affects_transformation_only_when_location_has_freshwater
    - hidden_cult_observer_not_revealed_before_visibility_trigger
  expected_events:
    - event_tide_turning
    - event_cult_observer_moves
  scoring_checks:
    - truth_preserved
    - self_preserved
    - horror_contained
```

Semantic Checks

Check	Failure example
Anchor preservation	A named bridge changes location without mutation.
Object continuity	The same key exists in two inventories.
Time coherence	Travel takes zero ticks despite route cost.
Rule authority	A deprecated rule overrides a canonical rule.
Hidden leakage	A private observer is named before discovery.
Partition boundary	Troupe A's private note appears in Troupe B's context.
Event causality	A consequence appears before prerequisite event.
Replay stability	Same seed and state produce different committed state.

Certification Report

```
certification_report:
  report_id: cert_dagon_path_001_run_04
  audit_pack_id: audit_dagon_path_001
  result: failed
  severity_counts:
    critical: 1
    major: 2
    minor: 5
  failures:
    - failure_id: fail_001
      severity: critical
      type: hidden_information_leakage
      turn: 7
      description: "Candidate prose revealed npc_cult_observer_three before visibility trigger."
      repair_hint: "Add hidden actor to denied fields in context packet."
    - failure_id: fail_002
      severity: major
      type: time_inconsistency
      turn: 9
      description: "Route consumed 0 ticks; route sheet requires 24 ticks."
```

Drift Velocity

Drift velocity is the rate at which contradictions accumulate under sustained play. It should be measured as failures per turn, weighted by severity. AGE should compare drift velocity against a baseline LLM-only run and against prior versions of AGE.

Replay Requirements

A replay record must include source version set, model profile, prompt template versions, tool versions, random seeds, initial state snapshot, request packets, scope packets, resolution packets, mutation packets, output envelopes, and verifier results. Without these, "replay" is only a transcript.

Acceptance Test

A certification test should run the same scenario under three conditions: direct LLM roleplay, AGE without verifier, and full AGE. The full AGE run should show fewer critical state contradictions and should identify failures with repairable packets.

White Paper 11 - Worked Dagon-Style Runtime Trace

This paper follows the same presentation burden used throughout this pass: define the subsystem, define its data model, describe normal execution, describe failure behavior, and state how to test it. The purpose is to make the subsystem buildable rather than merely plausible.

Abstract

The Dagon-style scenario is useful because it forces AGE to coordinate time, location, transformation, hidden observers, environmental effects, carried objects, and final scoring. This paper converts that kind of narrative sequence into runtime records.

Scenario Shape

The player begins on a coastal route, moves through a sea cave, experiences transformation pressure, crosses toward freshwater and elevation, interacts with hidden cult pressure, and receives final scores. The exact literary surface is less important than the technical stressors.

Initial State Snapshot

```
snapshot_id: snapshot_dagon_start
world_tick: 200
active_partition: partition_locus_boothbay_shoreline
actor_sheets:
  - actor_id: pc_marsh_reporter
    location_id: locus_boothbay_strand
    inventory_id: inv_pc_marsh_reporter
    state_tracks:
      exhaustion: 0
      transformation: 1
      horror: 1
object_sheets:
  - object_id: obj_crystalline_vertebra_fragment
    location_id: locus_boothbay_devils_throat_cave
    object_type: evidence_macguffin
location_sheets:
  - location_id: locus_boothbay_devils_throat_cave
    conditions:
      saltwater: true
      freshwater: false
      tide_sensitive: true
  - location_id: locus_boothbay_maiden_tear_footbridge
    conditions:
      freshwater: true
      elevation: ridge
hidden_events:
  - event_cult_observer_moves
```

Turn Trace

Turn	Player action	Runtime state operation	Visible result
1	Searches strand.	Reads shoreline Location Sheet and event table.	Finds signs of unnatural tide.
2	Enters cave.	Commits TransitionPacket to cave; advances ticks.	Cave description uses saltwater conditions.
3	Takes fragment.	Commits object transfer from location to inventory.	Fragment is now carried.
4	Studies marks.	CAL retrieves scenario lore and hidden constraints.	Reveals partial truth, not cult observer.
5	Leaves cave for ridge.	Commits route travel and tide event.	Time and exhaustion update.
6	Crosses footbridge.	Checks freshwater condition against transformation track.	Transformation stress increases.
7	Looks back.	Visibility check may reveal observer if conditions met.	Hidden actor remains hidden or becomes clue.
8	Chooses final action.	Computes final score from committed tracks.	Final assessment follows state.

Transition Example

```
transition_packet_id: trans_dagon_006
actor_id: pc_marsh_reporter
from_location: locus_boothbay_devils_throat_cave
to_location: locus_boothbay_maiden_tear_footbridge
route_id: route_cave_ridge_footbridge
start_tick: 218
end_tick: 242
crossed_events:
  - event_tide_turning
  - event_cult_observer_moves
state_effects:
  - set_location: pc_marsh_reporter = locus_boothbay_maiden_tear_footbridge
  - increment_track: pc_marsh_reporter.exhaustion +1
  - set_flag: pc_marsh_reporter.silver_tracery_visible = true
```

Final Score Packet

```
score_packet:
  score_id: score_dagon_final_001
  actor_id: pc_marsh_reporter
  scoring_basis:
    truth_preserved:
      from_flags: [evidence_collected, cult_pattern_understood]
      value: 1.7
    self_preserved:
      from_tracks: [transformation, exhaustion, horror]
      value: 2.4
    horror_contained:
      from_events: [observer_not_followed_to_town, fragment_secured]
      value: 1.3
  visible_summary_allowed: true
```

Why This Matters

The Dagon-style trace proves whether AGE can preserve a long causal chain. If the final score is merely inferred from prose tone, AGE has failed. If the final score is computed from committed state, AGE has become a game engine.

White Paper 12 - MVP Implementation Roadmap and Evaluation

This paper follows the same presentation burden used throughout this pass: define the subsystem, define its data model, describe normal execution, describe failure behavior, and state how to test it. The purpose is to make the subsystem buildable rather than merely plausible.

Abstract

The MVP should prove the state-authoritative loop in a bounded game. The product should not begin with unrestricted world generation, public marketplace scale, or professional role services. Those expansions depend on the core runtime.

Build Order

Phase	Build	Exit test
1	Property sheet schemas and event log.	Load, validate, mutate, and replay simple state.
2	DomainPartition and TickPolicy.	Scope local scene and advance time correctly.
3	Location, object, actor, inventory operations.	Move actor, transfer item, preserve versions.
4	AGEScript scene compiler.	Compile scene to deterministic artifact.
5	Input Coordinator and Scope Resolver.	Classify and scope 100 mixed inputs.
6	Rules Service and CAL over bounded corpus.	Answer rules with authority packets.
7	Fiction Constructor and Output Verifier.	Render scene prose without state contradiction.
8	Multiplayer spotlight and OCC conflict.	Resolve simultaneous object claim.
9	Event Scheduler and missed-event summaries.	Execute off-screen events and summarize on observation.
10	Semantic QA replay harness.	Certify Dagon-style trace against invariants.

Minimum Demo Content

The MVP needs one content pack, not a universe. The pack should include:

- one realm;
- three to five loci;
- ten to twenty locations or submaps;
- twenty to fifty objects;
- five to ten actors;
- three event tables;
- five AGEScript scenes;
- one rules subset;
- one Dagon-style stress path;
- one multiplayer conflict path;
- one semantic QA audit pack.

The first build should not implement a full world generator, a marketplace, arbitrary professional roles, third-party plugin ecosystem, unrestricted multi-modal generation, or large-scale public persistent worlds. Each of those can consume the project before the core loop is proven.

Engineering Non-Goals

Evaluation Metrics

Metric	Target for MVP
State-prose contradiction rate	Fewer than 1 major contradiction per 100 turns.
Replay state stability	Same seed and source versions produce same committed state.
Hidden leakage	Zero critical hidden-field leaks in audit scenarios.
Conflict handling	All OCC conflicts return structured policies.
Event correctness	Scheduled events fire at correct ticks or are explicitly deferred.
CAL authority	95 percent of bounded rules questions return correct authority tier.
Authoring friction	A trained author can create a small scene without editing raw database rows.

Staffing Implications

AGE needs system engineering before broad content staffing. The first team should cover backend state architecture, database design, LLM orchestration, game rules modeling, authoring UX, and QA harness construction. Creative writing is valuable, but it should not lead the architecture. The architecture must make writing safe to render.

Risk Register

Risk	Mitigation
Scope explosion	Keep MVP bounded to one content pack and one rules subset.
Authoring burden	Build forms over property sheets; do not expose raw YAML to ordinary users.
LLM drift	Use context packets, verifier, replay, and state-first rendering.
False determinism	Record source versions, prompt versions, model profile, seeds, and tool versions.
Sterile prose	Constrain facts, not voice. Permit sensory and stylistic variation after state is known.
Multiplayer confusion	Use spotlight windows, aggregation, and visible conflict policies.

Decision Gate

AGE is ready for broader investment only when the MVP can run the same stateful scenario longer and cleaner than an LLM-only baseline, while producing audit packets that explain every committed state change. If it cannot do that, adding more content or more roles will not solve the core problem.

Part II - Expanded Technical Detail

The preceding papers define the technical presentation standard and the core service contracts. This part carries forward the broader source-promotion rebuild from the prior pass. It is retained because it contains additional worked examples, schema fragments, and subsystem prose that remain useful for implementation review. The duplicated concepts are intentional: Part I is the tighter systems-paper presentation, while Part II gives a wider reference treatment for engineers and authors who need more context.

Part II - Expanded Source-Promotion Technical Detail

Definitions Used Before the Argument

The Amazing Game Engine [AGE] is a state-authoritative narrative engine. AGE uses generative prose, but it does not let generated prose become truth by itself. The engine keeps truth in structured state, validates proposed changes, commits only approved changes, and then asks a language model to render prose that conforms to the committed state.

A Large Language Model [LLM] is a model that generates or transforms text. In AGE, an LLM may propose narration, dialogue, summaries, classifications, and possible state deltas. An LLM does not directly advance time, change location, create inventory, resolve conflicts, or commit truth.

A Large Model System [LMS] is the larger operating arrangement around one or more language models. An LMS may include retrieval, graph search, tools, task routing, workspace access, adapters, and audit controls. The Large Model System Graph [LMS-Graph] is the maintained graph and relational corpus substrate inside AGE. It stores sources, concepts, rules, examples, authority tiers, versions, dependencies, conflicts, and structured facts.

The Corpus Arbitration Layer [CAL] is the AGE service that answers questions against LMS-Graph. CAL retrieves evidence, weighs authority, exposes conflicts, preserves source provenance, and identifies when a human ruling is required. CAL is used by the Rules Service, authoring tools, and quality checks.

A DomainPartition is the principal runtime partition object. It is a bounded simulation and narrative jurisdiction with owned state, a tick policy, access rules, bridge rules, validation rules, and commit semantics. DomainPartition derives from the more general AbstractPartition contract. Human-readable partition labels use a colon delimiter such as ConcernPartition:Economy. Code may use typed fields or generics rather than literal colon names.

A property sheet is a structured record that declares the state surface of an entity. A property sheet may describe a location, object, actor, resource pool, event, role, rule, or partition. The property sheet records identity, scope, owner, version, permitted mutations, dependencies, and any fields required for the active subsystem.

AGEScript is the authored scenario and scene language for AGE. AGEScript defines scenes, spotlights, resolution rules, state mutations, plot progress, and narrative hooks. AGEScript source is readable by authors but compiles to deterministic JSON for runtime execution.

A troupe is a small group of players sharing a local narrative branch. Troupe-local partitions are isolated from other troupe-local partitions. Cross-troupe influence travels through redacted ripple events routed by shared world partitions, not through direct shared scenes.

Optimistic concurrency control [OCC] is the conflict-control method used when multiple clients may attempt to change the same state. A client writes against a known version. If the version has changed, the write does not silently overwrite state. AGE either retries, rejects, or resolves the conflict through a deterministic game procedure.

Retrieval-Augmented Generation [RAG] is the common practice of retrieving source material and placing it into a model context. AGE may use Retrieval-Augmented Generation at a partition boundary, but partitioning is not RAG. Retrieval asks what material should be injected into context. Partitioning decides where truth lives, who can change it, what can cross a

boundary, and how a change becomes authoritative. Retrieval is also context-expensive, so structured state reads, identifiers, projections, and cached summaries should be preferred whenever they can answer the request.

White Paper Standard Used Here

These white papers are written to the standard implied by technical systems papers rather than ordinary product briefs. Each major subsystem is explained by problem, target environment, data model, execution flow, failure mode, and implementation test. A diagram or table is not treated as a replacement for prose. A table summarizes the system after the reader has been given the system.

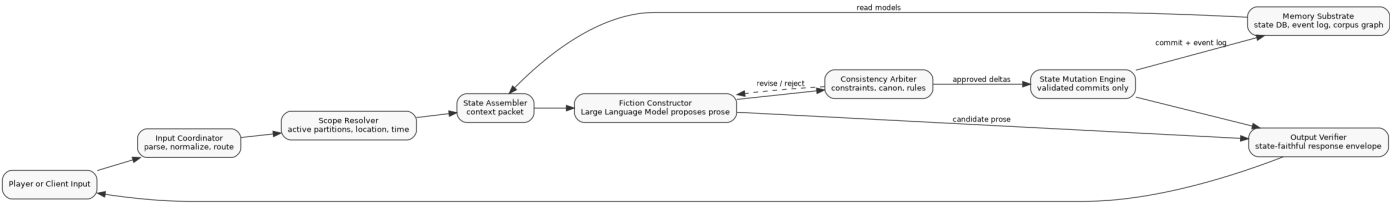
The comparison model is straightforward. A useful systems paper defines the abstraction, shows the architecture, gives concrete data structures or interfaces, explains why the design decision exists, describes normal execution, describes failure behavior, and gives enough implementation detail that an engineer can start building or challenging the design. AGE needs the same treatment.

White Paper 1 - Core Runtime and State Authority

Purpose

The AGE core runtime exists to answer one question: what is true after a player or system action? A generative model can write plausible prose, but prose is not sufficient authority. A game world needs state that can be read, checked, changed, replayed, and audited. The runtime therefore separates input, scope, context assembly, language-model generation, consistency arbitration, state mutation, persistence, and output verification.

This paper defines the runtime as a transaction pipeline. The pipeline should be treated as the first implementation target because every later subsystem depends on it.



Target Environment

The first AGE target environment is a bounded role-playing game world with one to five players per troupe, structured locations, object and inventory tracking, event calendars, authored scenes, and a rules corpus. The system should support long-running play across hundreds of meaningful decisions, but it should not begin by trying to support unbounded worlds or thousands of players in one narrative thread.

The target environment has several properties that determine the architecture.

Environment property	Consequence for the runtime
Player input is natural language.	The Input Coordinator must classify and normalize before state access.
The world has persistent objects.	Inventory, containers, locations, and resources require versioned property sheets.
Time matters.	Time advancement is owned by a World Time Authority, not by prose.
Players can act concurrently.	The State Mutation Engine needs OCC and conflict policies.
The model can hallucinate.	The Fiction Constructor proposes, but does not commit.
Rules may be ambiguous.	CAL must surface source conflicts and preserve human rulings.

Runtime Components

The Input Coordinator receives raw input and classifies it. A message may be a player action, command, rules question, authoring command, out-of-character instruction, safety issue, or system-management request. The component should be deterministic after any optional classification step. The system cannot allow an LLM to choose freely whether input is an in-world action or a state-editing command.

The Scope Resolver determines the active state boundary. It identifies the player character, troupe, active ScenePartition, LocusPartition, RealmPartition, active concern overlays, relevant rules, visible entities, current tick, scheduled events, and any output mode requirements. Scope resolution is the first point where partition authority appears.

The State Assembler builds a context packet. A context packet is a structured input to the Fiction Constructor. It contains selected facts, not raw database access. The packet should include visible actors, visible objects, location state, current time, applicable event hooks, active rules, hidden constraints expressed as non-disclosure instructions, and facts that must not be contradicted.

The Fiction Constructor is the LLM-facing renderer. It writes candidate prose and may propose state deltas. A candidate delta is not trusted merely because it appears in prose. The model may write, "You take the key," but the object transfer must still pass validation and commit.

The Consistency Arbiter validates candidate deltas and candidate prose. It applies rule checks, corpus checks, partition checks, source checks, safety checks, and state consistency checks. It may use an LLM for semantic contradiction detection, but hard boundaries must be deterministic.

The State Mutation Engine commits approved deltas. It performs atomic updates, increments versions, appends to the event log, writes replay metadata, and emits events. It must be non-generative.

The Memory Substrate stores canonical state, event history, corpus graph material, embeddings, and replay records. The store may be physically implemented as several databases, but the runtime should expose a controlled state interface rather than letting components write freely.

The Output Verifier checks that the visible response matches committed state. If the text claims that an object moved, an event occurred, a rule resolved, or time advanced, the verifier should be able to point to the committed mutation or reject the output.

The Proposal-Validation-Commit Contract

AGE state changes must follow a proposal-validation-commit contract.

1. A player, author script, event schedule, or subsystem proposes a change.
2. The proposal declares the target entity, target property, operation, expected version, cause, rule source, and partition authority.
3. The validator checks schema, authority, preconditions, invariants, source rules, and concurrency version.
4. The mutation engine commits the change or returns a failure result.
5. The event log records the result.
6. The Fiction Constructor renders the outcome after the result is known.

A proposal should resemble this:

```
proposal_id: prop_00412
cause:
  input_turn: turn_000184
  actor_id: pc_adel
  action_class: take_object
target:
  entity_type: object
  entity_id: item_iron_key_orren
operation:
  type: transfer_object
  from_container: inv_npc_bartender_orren
  to_container: inv_pc_adel
preconditions:
  - actor_within_reach: true
  - object_visible_or_known: true
  - from_container_version: 17
authority:
  partition_id: scene_blackstone_tavern_03
  rule_source: rules_inventory_transfer_v01
resolution:
  required_test: stealth_vs_notice
  deterministic_seed: campaign_7a3f9b:turn_000184
```

The commit should resemble this:

```
commit_id: commit_00991
proposal_id: prop_00412
committed_at_tick: 10489
mutations:
- entity_id: item_iron_key_orren
  field: container_id
  old_value: inv_npc_bartender_orren
  new_value: inv_pc_adel
- entity_id: inv_npc_bartender_orren
  field: version
  old_value: 17
  new_value: 18
- entity_id: inv_pc_adel
  field: version
  old_value: 41
  new_value: 42
events_emitted:
- item_transferred
- possible_theft_observed
replay:
seed: campaign_7a3f9b:turn_000184
rules_version: rules_mvp_0_1
model_context_packet: ctx_00244
```

Output Envelope

A player-facing response is only one layer of the output. Internally, AGE should produce an output envelope. The envelope separates committed truth from narration.

```
output_envelope_id: out_000184
visible_text: |
  Orren jabs a finger toward the nearest guard. In that instant, your hand slips beneath the bar towel and closes around cold iron. The key vanishes into your sleeve before he turns back.
state_commit_ids:
- commit_00991
world_tick_after: 10489
visible_state_changes:
- "Iron key added to Adel's inventory."
hidden_state_changes:
- "possible_theft_observed event emitted to tavern watcher queue."
forbidden_claims_checked:
- "Do not claim Orren noticed unless notice event commits."
- "Do not reveal hidden watcher identity."
next_prompt_hooks:
- "key is now in Adel inventory"
- "bartender remains distracted"
```

The output envelope is also the testing surface. Semantic QA should compare visible claims to commit records.

Failure Modes

A prose-first system fails when it changes truth without a state update. AGE prevents that by rejecting output or forcing a state transaction.

A state-first system can still fail if it over-constrains prose. The mitigation is to constrain facts and consequences, not diction. The Fiction Constructor should have wide freedom in sensory detail, pacing, and dialogue as long as it does not alter truth.

Another failure mode is invisible recursion. If partitions or tools call each other directly, the system can create hidden state changes. AGE should require routed events and proposals. Direct writes by LLM-facing components should be prohibited.

A fourth failure mode is schema sprawl. If every subsystem invents its own entity format, the engine becomes unmaintainable. Property sheets solve this by giving every entity a common identity, scope, version, authority, and mutation envelope.

Implementation Tests

The first runtime test should be a deterministic inventory transfer. The same seed, input, actor state, object state, and rule version should produce the same commit or the same conflict.

The second test should be a rejected hallucination. Force the Fiction Constructor to claim that a locked door opens without a successful unlock proposal. The Output Verifier must reject the claim or rewrite it.

The third test should be a concurrent object claim. Two clients submit object-transfer proposals against the same container version. The system should detect the version conflict and call the configured conflict policy.

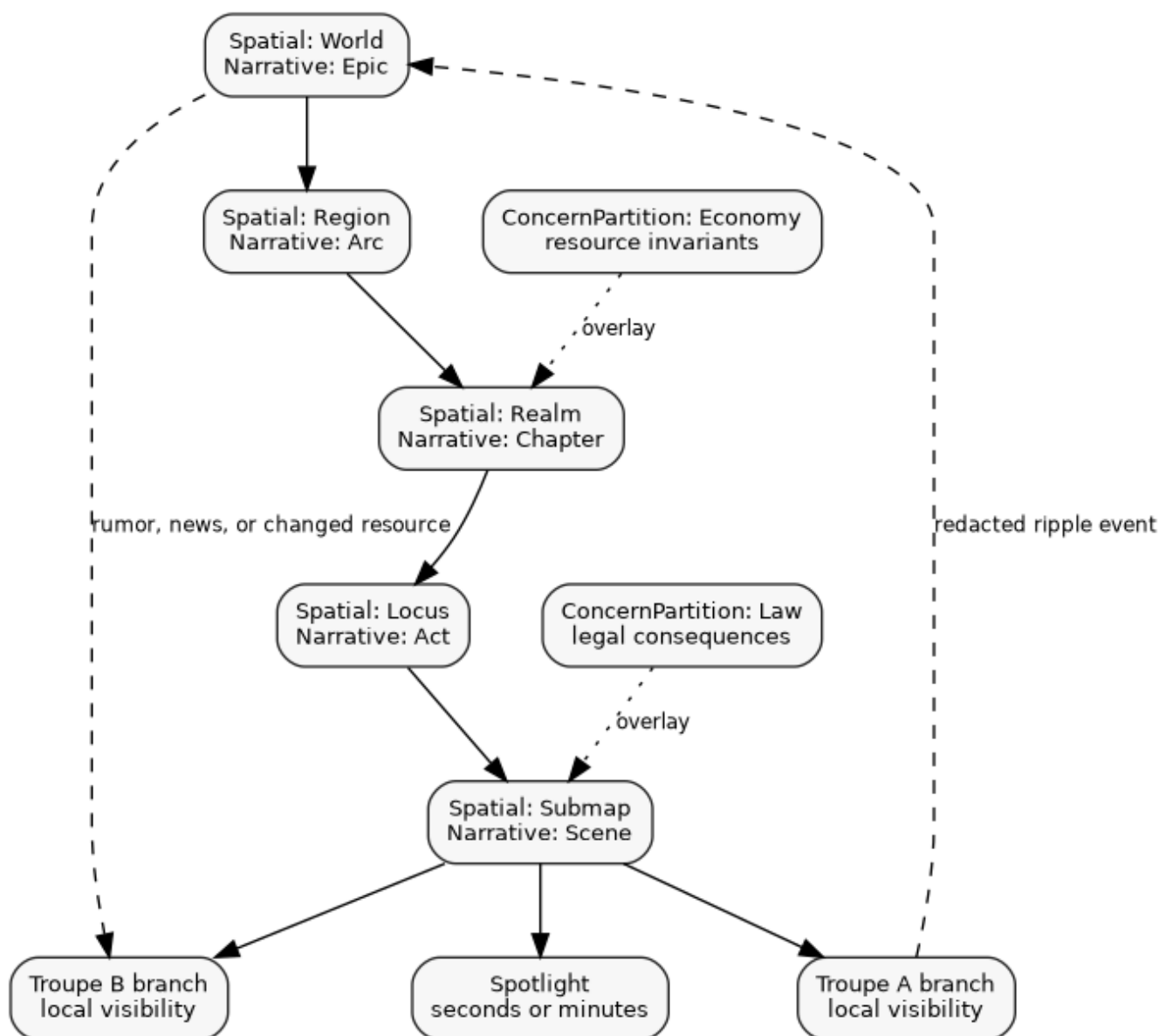
The fourth test should be a missed event. Advance time during player travel across a scheduled event and verify that the event commits or is summarized according to its missed-event policy.

White Paper 2 - DomainPartition, AbstractPartition, and Database Partitioning

Purpose

Partitioning is the load-bearing abstraction of AGE. It is how the engine decides where truth lives, who can see it, who can change it, how time advances, and how consequences move between local play and wider world state. A partition is not merely a namespace, folder, shard, or retrieval filter. It is a bounded simulation and narrative jurisdiction.

The foundational contract is AbstractPartition. The principal runtime implementation is DomainPartition. DomainPartition is burdened with state ownership, proposal flow, event routing, tick scheduling, bridge policy, validation policy, determinism policy, and audit metadata. Specialized behavior should mostly be supplied by facets and policies rather than deep inheritance trees.



The AbstractPartition Contract

AbstractPartition is the minimal interface shared by all future partition species. It should not contain world logic. It should define identity, event ingress and egress, proposal handling, commit, and rollback or compensation support.

```
public abstract class AbstractPartition {
    protected final PartitionId id;
    protected final String name;

    public abstract void receive(Event event);
    public abstract List<Event> drainOutboundEvents();

    public abstract void propose(Proposal proposal);
    public abstract Commit commit(Map<String, String> metadata);
    public abstract void rollback(CommitId commitId);
}
```

The point of this contract is not Java specifically. The point is the rule: no component should mutate a partition without using the proposal and commit path.

A DomainPartition should contain:

DomainPartition as the Principal Runtime Object

```
partition_id: part_realm_veridian_chapter2
name: "Veridian Duchy / Chapter 2"
facets:
  spatial: Realm
  narrative: Chapter
  concerns:
    - Economy
    - Law
topology:
  parent: part_region_archipelago_arc1
  children:
    - part_locus_blackstone_pass_act4
    - part_locus_sunken_harbor_act2
  adjacent:
    - part_realm_northmarch_chapter1
ownership:
  troupe_owner: null
policies:
  tick_policy: realm_daily
  bridge_policy: region_ripple_redacted
  validation_policy: realm_state_invariants
  determinism_policy: hard_for_state_soft_for_prose
state_surface:
  read_models:
    - realm_politics
    - realm_trade
    - realm_known_events
  write_models:
    - faction_goal_progress
    - realm_flags
    - scheduled_events
versions:
  state_version: 288
  schema_version: partition_schema_0_2
```

This record is a database partition, a scheduling unit, and a semantic firewall. It is also the point at which retrieval becomes subordinate to state authority. A retrieval result may be relevant, but it consumes context and cannot bypass the partition's access rules or commit rules.

Facets Instead of Taxonomy Explosion

The origin branch proposes SpatialPartition, NarrativePartition, ConcernPartition, and RolePartition. The important design refinement is that these should be facets or specifications where possible. A ScenePartition may have spatial, narrative, concern, and role properties at the same time. If every combination becomes a subclass, the class graph becomes unbuildable.

A partition instance can therefore carry several facets:

```
partition_id: part_scene_trial_of_salt_01
facets:
  spatial: Locus
  narrative: Scene
  concern: Law
  role: RulesArbiter
```

This says that the partition is a scene in a local place, governed by legal constraints, and possibly mediated by a Rules Arbiter role. It does not require a class named LawSceneAtLocusRulesArbiterPartition.

Spatial and Narrative Alignment

AGE uses two aligned ladders.

Spatial scope	Narrative scope	Typical tick policy	Typical state
World	Epic	Monthly or seasonal	climate, civilization-scale events, global calendars.
Region	Arc	Daily or weekly	faction campaigns, trade routes, migrations.
Realm	Chapter	Daily	settlements, politics, war fronts, resource flows.
Locus	Act	Hourly or scene-driven	local quests, non-player character webs, resource pools.
Submap or room	Scene or Spotlight	Minutes or action turns	immediate actors, objects, hazards, dialogue, tests.

The alignment is useful, but it must not be forced into inheritance. A narrative arc can span several regions. A world event can manifest in one local scene. A law concern can overlay a tavern scene and a regional court case.

Database Partitioning

AGE database partitions should follow the authority model. The physical database can use relational tables, document records, graph edges, event logs, and vector indexes, but the write path should be organized around partition authority.

A practical storage layout is:

Store	Purpose	Partition relationship
State database	Current property sheets and active versions.	Sharded or indexed by partition_id and entity_id.
Event log	Immutable committed events and proposals.	Appended by partition authority.
Replay store	Seeds, prompt versions, model versions, source versions, outputs.	Linked to commits and turns.
Corpus graph	Rules, sources, citations, concepts, conflicts.	Queried by CAL and Rules Service.
Vector store	Semantic retrieval for sources and summaries.	Partition-scoped retrieval only unless bridge policy allows broader retrieval.
Projection store	Read models for fast context assembly.	Generated from authoritative state and event logs.

The physical database can be federated by world instance. A world instance containing 100 to 1,000 players should not share writable local state with another instance. A meta-world service may track instance membership and social grouping, but local story truth should remain per-instance.

A bridge policy decides whether an event can cross from one partition to another and how it must be transformed. Troupe boundaries require especially strong bridge rules.

Bridge Policies

Event classes should include at least:

Event class	Meaning	Cross-boundary behavior
LOCAL	Ordinary local consequence.	Does not leave the local partition.
RIPPLE	Consequence that may affect a wider scope.	May move to a coarser partition after redaction.
GLOBAL_CONTROL	Administrative or system event.	Restricted to system actors and audit.

A local event such as "Borin insults the innkeeper" should remain local unless the insult has faction or legal consequence. A ripple event such as "the bridge over the war road is destroyed" may move upward to a realm or region partition. Other troupes should receive only redacted consequences such as "traffic is diverted after a bridge collapse" unless their characters have direct evidence.

```
ripple_event:
  event_id: evt_bridge_destroyed_001
  class: RIPPLE
  source_partition: part_scene_bridge_ambush_troupe_a
  payload_private:
    actor_ids: [pc_borin, pc_adel]
    exact_location: bridge_hex_14_9
    method: alchemical_fire
  payload_world:
    public_fact: "The old war-road bridge has collapsed."
    region: north_road
    confidence: 0.8
    earliest_report_tick: 11820
    bridge_policy: redact_private_actors_and_method
```

Context-Budgeted Retrieval as a Subordinate Aid

Retrieval should be treated as a read-only, scoped, budgeted, and optional aid. Each retrieval call should declare the partition, corpus scope, source type, time validity, context budget, and whether retrieved facts may affect state.

```
retrieval_request:
  partition_id: part_scene_trial_of_salt_01
  corpus_scope: rules_and_local_law
  question: "What penalty applies to theft from a temple storehouse?"
  allowed_sources:
    - rules_core
    - realm_law_veridian
  time_valid_at_tick: 11800
  write_authority: false
```

The response becomes evidence for a proposal or answer. It does not become a mutation. This distinction prevents retrieval soup: the condition where a model receives a relevant chunk and treats it as permission to rewrite the world.

Failure Modes

Partition systems fail when they allow hidden calls. A partition should emit events and proposals, not call another partition's private mutator.

They also fail when they confuse type with policy. A spatial partition can be hard or soft. A concern partition can be hard or soft. A role partition can be deterministic or exploratory. Policy must be attached explicitly.

A third failure mode is leakage across troupes. If raw transcripts or secrets from troupe A enter troupe B's context, the model may reveal or act on them. Bridge policy and state assembly must prevent that.

Implementation Tests

The first partition test should create two troupe-local scenes and a shared world hub. A LOCAL event emitted by troupe A must not reach troupe B. A RIPPLE event must reach the world hub in redacted form and reach troupe B only as a rumor or news packet.

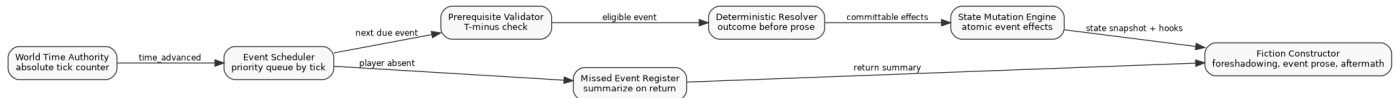
The second test should verify partition-scoped retrieval. A role inside a scene should retrieve only allowed rules and local facts. It should not retrieve hidden state from another troupe.

The third test should verify tick policy. A scene partition can advance minutes while a world partition advances monthly. The engine must not require every partition to tick at the same rate.

White Paper 3 - Time, Location, and Locale-to-World Movement

Purpose

Time and location are first-class state in AGE. The engine cannot rely on narrative text such as "later that evening" or "after a long walk" unless a state system records what time passed, what route was used, what events were crossed, and what changed during the transition. This paper defines the World Time Authority, event calendar, temporal cohesion window, location sheets, and transition packets.



World Time Authority

The World Time Authority owns the absolute tick counter. A tick is the smallest unit of game time used by the scheduler. The first MVP can use five-minute ticks because that is fine enough for travel, investigation, watch schedules, and environmental change while remaining coarse enough for efficient scheduling. Combat or tactical scenes can use sub-turns inside a tick if needed, but those sub-turns should reconcile back to the tick record.

```
world_time_authority:
  epoch_calendar: sarna_len_example_epoch
  epoch_datetime: "1247-03-15T09:00:00"
  tick_duration_seconds: 300
  current_tick: 10488
  rules:
    - players_do_not_set_ticks
    - language_models_do_not_set_ticks
    - scheduler_advances_ticks_after_action_resolution
    - time_compression_processes_due_events
```

The World Time Authority publishes time_advanced events. Partitions subscribe according to tick policy. A scene partition may handle every tick. A realm partition may handle one daily tick. A world partition may handle one monthly tick.

An event calendar is a priority queue of scheduled and triggered events. Each Event Sheet declares schedule, scope, prerequisites, effects, narrative seeds, and missed-event policy.

Event Calendar

```
event_id: meteor_strike_xontis
scope_level: arc
scheduled_tick: 10450
location:
  region_id: region_xontis
  coordinates: {x: 245, y: 187}
prerequisites:
  - prophecy_fulfilled: true
  - player_presence: any_within_50km
effects:
  immediate:
    - create_location: locus_meteor_crater_xontis
    - set_flag: prophecy_fulfilled
    - damage_radius: 5km
  delayed:
    - tick_offset: 1440
      set_flag: strange_crystals_emerge
narrative_seeds:
  pre_event:
    - "strange lights over Xontis"
    - "astrologers warn of celestial fire"
  during_event:
    - "the sky tears open"
  post_event:
    - "a smoking crater steams where the forest stood"
missed_event_policy: summarize_on_next_relevant_entry
```

The event outcome must be resolved before prose. The Fiction Constructor can foreshadow, dramatize, and summarize, but it cannot decide whether the meteor strikes if the scheduler and prerequisite validator have already resolved the event.

Temporal Cohesion Window

Spatially separated stories must not drift beyond plausible travel time. If Realm A and Realm B are three weeks apart by foot, related arc events should remain within a cohesion window derived from that travel time. This is not because every rumor travels instantly. It is because a shared world becomes incoherent if realm-level events are months apart while the map says they are weeks apart.

```
def validate_temporal_cohesion(return_tick, event_tick, travel_weeks):
    max_drift = travel_weeks * 7 * 24 * 12 # five-minute ticks
    actual_drift = abs(return_tick - event_tick)
    if actual_drift <= max_drift:
        return "cohesive"
    if actual_drift <= max_drift * 1.5:
        return "drifting_requires_reconciliation"
    return "broken_requires_referee_or_system_reset"
```

The cohesion window allows absence and travel to matter. A player returning to a distant realm may receive a missed-event summary. The summary should be anchored to actual ticks and routes.

Time Compression

Players do not need to experience every tick. The system can compress time during travel, downtime, or absence. Compression is not a skip. It is a controlled advancement that checks scheduled events, resource regeneration, faction actions, and player commitments.

Activity	Compression behavior	Required checks
Active scene	Tick or sub-turn resolution.	Actor actions, object changes, immediate hazards.
Travel	Advance by route time.	Route events, weather, pursuit, carried objects, scheduled events.
Downtime	Advance by declared task blocks.	Resource costs, skill progress, faction events, obligations.
Absence	Advance by world policy.	Missed events, changed relationships, decay, opportunities lost.

A travel compression must produce a transition packet.

Location Sheets

A Location Sheet is the authoritative state record for a place. It should not be replaced by prose.

```
location_id: locus_boothbay_devils_throat_cave
location_type: cave_interior
parent_location: locus_boothbay_shoreline
partition_id: part_locus_boothbay_act_02
coordinates:
  map: boothbay_region
  x: 41
  y: 12
exits:
  - route_id: route_cave_to_shoreline
    to_location: locus_boothbay_shoreline
    base_travel_ticks: 3
    constraints: [low_tide_required]
  - route_id: route_cave_to_hidden_pool
    to_location: locus_hidden_tidal_pool
    base_travel_ticks: 2
    hidden_until_flag: cave_symbols_deciphered
conditions:
  tide_state: turning
  light_level: dim
  surface: wet_stone
  contamination: violet_sheen_active
active_event_tables:
  - event_table_tidal_cave_phenomena
contained_objects:
  - item_crystalline_vertebra_fragment
visibility:
  public_description_level: partial
  hidden_observers_possible: true
version: 22
```

This sheet tells the State Assembler what to include. It also tells the transition system what can be done from the location.

Locale-to-World Shift

A locale-to-world shift happens when a player action moves from the immediate scene into a wider spatial or narrative scope. Leaving a tavern for a city street, traveling from a harbor to a lighthouse, or moving from a local investigation to a regional faction consequence are all shifts. The engine needs a transition packet to prevent continuity loss.

```
transition_packet_id: trans_00103
actor_ids:
  - pc_marsh_reporter
from_location: locus_boothbay_devils_throat_cave
to_location: locus_maiden_tear_footbridge
route_id: route_cave_ridge_footbridge
from_partition: part_scene_cave_escape
to_partition: part_locus_boothbay_night_act
start_tick: 218
end_tick: 242
travel_time_ticks: 24
route_events_checked:
  - tide_turning
  - cult_observer_patrol
  - freshwater_distance_effect
state_effects:
  - increase_track: exhaustion +1
  - set_flag: left_cave_after_tide_turn
  - update_condition: transformation_visible_on_left_hand
objects_carried:
  - item_crystalline_vertebra_fragment
missed_events: []
visibility_changes:
  - hidden_observer_status: possible_not_confirmed
```

The visible prose can describe gulls, brackish air, or the geometry of rooftops. The transition packet records what must remain true.

Dagon Scenario as a Stress Test

The Dagon example in the origin branch is a good early scenario for time and location because it stresses several engine responsibilities.

The player begins at harbor docks at 3:47 PM. Several choices advance time by approximately 12 to 38 minutes. The character moves through shoreline, cave entrance, cave interior, freshwater stream, ridge path, footbridge, and ravine confluence. The tide turns. Nightfall and moonrise matter. Freshwater changes the character's transformation state. A crystalline fragment changes temperature. The violet contamination progresses. Hidden cult pressure closes options. The player's body becomes an object of state change.

A prose-only system will remember these as atmosphere. AGE must promote them to state. Each location has a sheet. Each transition has time cost. Each transformation has a track. Each object has state. Each environmental exposure creates effects. Each missed or delayed action can close options.

Time fails when the model narrates time passage without a tick commit. Location fails when the model moves characters without a transition packet. Event calendars fail when scheduled events are discovered only after the prose needs them. Travel fails when route time is treated as flavor rather than state.

Failure Modes

The mitigation is strict ownership. The World Time Authority owns time. Location Sheets own place state. Transition packets own movement. Event Sheets own scheduled consequences. The Fiction Constructor describes all of them after they are assembled.

Implementation Tests

The first test should move a character through three locations with fixed travel times and verify that `world_tick_after` equals the sum of committed route costs.

The second test should schedule an event during travel and verify that compression pauses or records a missed event according to policy.

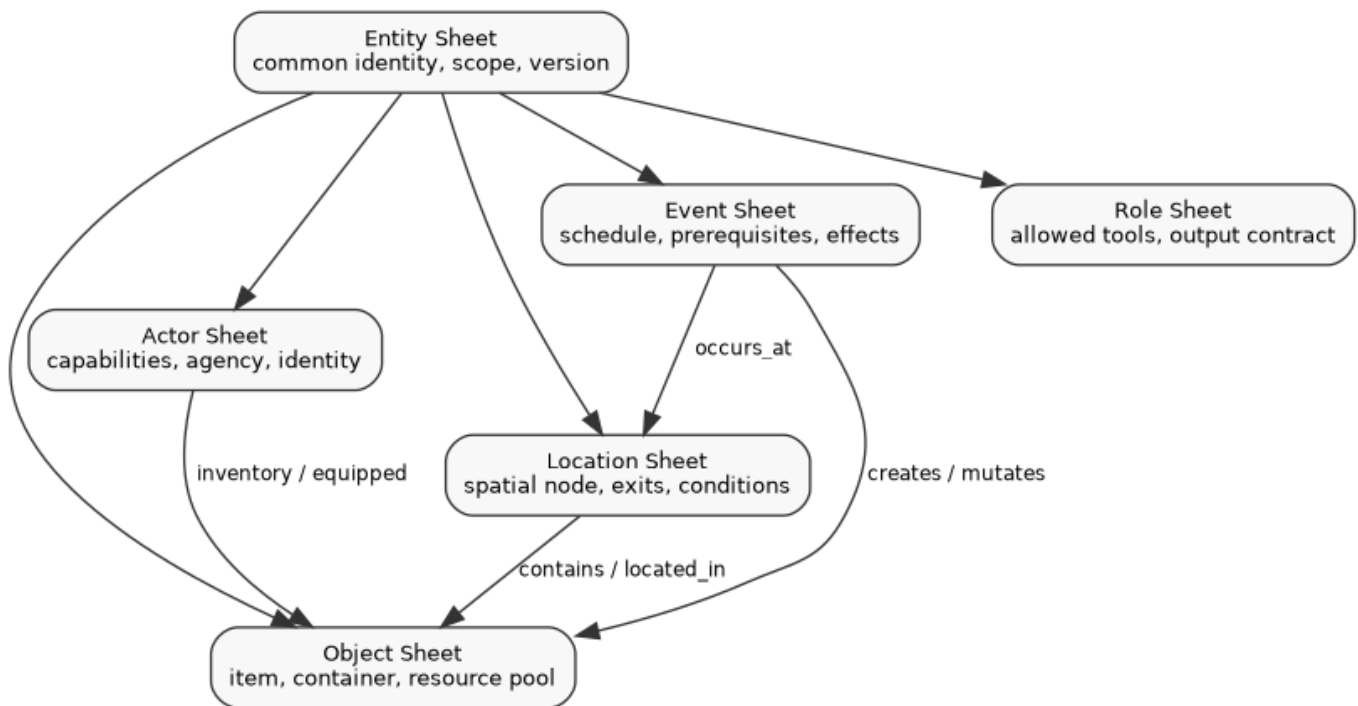
The third test should attempt an impossible transition, such as entering a cave route that requires low tide when the tide flag is false. The transition must fail before prose.

The fourth test should replay the Dagon route with the same seed and verify that location state, time, carried objects, and transformation track are preserved.

White Paper 4 - Property Sheets, Object Types, Actors, and Resources

Purpose

AGE needs a practical data surface. The origin branch contains inventory tracking, resource tracking, MacGuffin integration, actor ontology, roles, personas, identities, and object examples. These must be collected into a property-sheet system. A property sheet is the structured record by which AGE knows what exists and how it can change.



Common Entity Fields

Every property sheet should share a common header. This lets tools, partitions, audit reports, and replay systems reason across object types.

```
entity_header:
  entity_id: uuid_or_slug
  entity_family: actor | location | object | event | partition | role | rule
  display_name: string
  owning_partition: partition_id
  scope_level: character | scene | act | chapter | arc | epic | world
  authority:
    created_by: system | author | runtime | player_action
    mutable_by:
      - state_mutation_engine
    source_ref: optional_source_id
  version:
    schema_version: sheet_schema_0_1
    state_version: integer
    last_commit_id: commit_id
  visibility:
    player_visible: true_or_false
    hidden_fields:
      - list_of_fields
```

This common header allows every subsystem to ask the same first questions. What is the entity? Where does it belong? Who can mutate it? What version is current? What is visible to the player?

Actor Sheets

The actor ontology separates capability, agency, and identity. An assistant, role, persona, identity, character, non-player character, faction, and player-controlled character are not the same thing.

An Actor Sheet should record the actor's capability class, agency class, narrative identity, persona vector or trait model, current location, inventory container, permissions, and active obligations.

```
actor_id: pc_adel
entity_family: actor
actor_type: player_character
capability_class: embodied_character
agency_class: player_initiated
identity:
  public_name: Adel of Nareth
  social_tags: [mercenary, foreigner]
  known_reputations:
    blackstone_tavern: suspicious
persona:
  courage: 0.62
  caution: 0.48
  greed: 0.35
state:
  location_id: locus_blackstone_tavern_common_room
  inventory_container: inv_pc_adel
  conditions: [tired]
  resources:
    health: 18
    resolve: 7
permissions:
  can_initiate_actions: true
  can_commit_state: false
  can_propose_state: true
version: 42
```

The key rule is that an actor may propose action but does not directly commit state unless it is a system actor with explicit permission.

Role Sheets

A Role Sheet is not a character sheet. It is a behavioral contract for a Role Service. The role may be a Rules Arbiter, Fiction Constructor, Sports Therapist, Broadway Thespian, or other bounded assistant type. Role Service is a better product term than using Roles-as-a-Service as a universal umbrella.

```
role_id: role_rules_arbiter_mvp
entity_family: role
role_type: rules_arbiter
purpose: "Answer rules questions from the maintained game corpus."
allowed_tools:
  - cal_query
  - rules_table_lookup
  - cite_source
forbidden_actions:
  - mutate_world_state
  - invent_rule_when_source_conflicts
  - reveal_hidden_state_without_visibility_grant
output_contract:
  must_include:
    - interpreted_question
    - rule_answer
    - source_authority
    - human_decision_required
  must_not_include:
    - uncited_corpus_claims_when_source_available
determinism_policy: hard
semantic_audit_pack: audit_rules_arbiter_0_1
```

Object Sheets

Object Sheets cover items, containers, resource pools, and MacGuffins. The origin branch correctly treats inventory and resources as first-class world state, not prose memory.

```
object_id: item_iron_shortsword_001
entity_family: object
object_type: item
scope_level: character
owner_id: pc_adel
location:
  spatial_node_id: locus_blackstone_tavern_common_room
  container_id: inv_pc_adel
properties:
  name: Iron Shortsword
  weight: 1.2
  durability:
    current: 45
    max: 50
  traits: [sharp, balanced]
  value: 25
flags:
  is_macguffin: false
  plot_relevance: low
version: 9
```


A Container Sheet is a specialized object sheet.

```
object_id: chest_black_oak
entity_family: object
object_type: container
location:
  spatial_node_id: room_tavern_cellar
capacity:
  max_weight: 60
  max_slots: 20
lock_state:
  locked: true
  key_object_id: item_iron_key_orren
contents:
  - item_healing_potion_003
  - item_sword_of_kings
version: 14
```

Resource Pool Sheets

A resource pool is an object that represents countable or flowing resources. It may be depleted, regenerate, trigger events, or drive faction goals.

```
resource_id: iron_ore_blackstone_mine
entity_family: object
object_type: resource_pool
scope_level: act
location: locus_blackstone_pass
pool_type: depletable
current_units: 87
max_units: 200
depletion_rate: 1
regeneration:
  method: timed
  rate: 5
  interval_ticks: 1440
triggers:
  - when: current_units <= 20
    event: mine_nearly_depleted
    sets_flag: locus_economic_crisis
  - when: current_units == 0
    event: mine_exhausted
    triggers_plot: resource_war
version: 31
```

Resources can exist at different scopes. Torches in a room are scene resources. Ore in a mine is act or locus state. Timber forests and trade routes are realm resources. Strategic iron deposits or magical nexuses may be arc or world resources.

MacGuffin Sheets

A MacGuffin is a plot-driving asset with explicit contracts. It should not merely be a named object in prose.

```
object_id: sword_of_kings
entity_family: object
object_type: item
is_macguffin: true
location:
  container_id: chest_black_oak
plot_bindings:
  - plot_id: coronation_crisis
    trigger_conditions:
      - location: realm_capital_city
      - bearer: pc_or_royal_heir
    resolution_effects:
      triumphant:
        flags_set: [legitimate_heir_crowned, civil_war_averted]
      tragic:
        flags_set: [usurper_takes_throne, realm_fractured]
  - plot_id: artifact_corruption
    trigger_conditions:
      - durability_lte: 10
      - location: region_shadow_fell
    resolution_effects:
      corrupted:
        flags_set: [sword_corrupted, bearer_possessed]
version: 14
```

The MacGuffin sheet is what lets the Plot Engine know that moving the sword matters.

Location and Container Hierarchy

Containers are spatial nodes when they can hold state. A room can contain a chest. A chest can contain items. A player character can have an inventory container. A market stall can have a stock container. This allows proximity, visibility, theft, restocking, and depletion to use the same basic machinery.

```
World
  Region: Archipelago
    Realm: Veridian Duchy
      Locus: Blackstone Pass
        Building: Tavern of Weeping Stone
          Room: Cellar
            Container: Locked Oak Chest
              Item: Healing Potion
              Item: Sword of Kings
            Actor: Bartender Orren
              Container: Orren Inventory
                Item: Iron Key
          Scene: Canyon Ambush Site
            Resource Pool: Scattered Supplies
```

Mutability and OCC

Every object or container that can be changed should have a version. A client or subsystem writes against the version it read. If the version changed, the write must be revalidated or converted into a conflict procedure.

```
def transfer_object(object_id, from_container, to_container, expected_from_version):
    current = state.get_version(from_container)
    if current != expected_from_version:
        return Conflict(type="version_mismatch", policy="object_claim_resolution")
    state.move(object_id, from_container, to_container)
    state.increment_version(from_container)
    state.increment_version(to_container)
    event_log.append("object_transferred")
```

AGE should prefer narrative-integrated conflict resolution when the fiction supports it. Two characters grabbing the same sword can roll or compare established position. A shopkeeper restocking while a player steals may create a caught-in-the-act event. A resource pool depleted mid-harvest may yield partial success.

Failure Modes

The main failure is treating inventory as prose. Once that happens, the model can forget who has what, create duplicate objects, or spend resources that no longer exist. Another failure is treating resources as resettable scene dressing. Persistent worlds require resources to persist, deplete, regenerate, and trigger consequences.

A third failure is overloading actor, role, persona, and character. A Role Service contract is not a non-player character. A persona vector is not a legal identity. A character may have a persona and identity, but those are different state surfaces.

Implementation Tests

The first test should transfer an item between containers and verify versions.

The second test should deplete a resource pool, trigger a low-resource event, advance time, and regenerate according to the sheet.

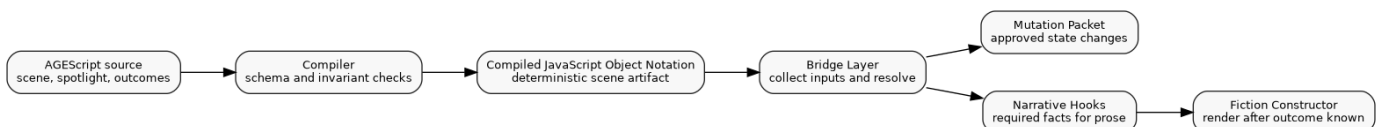
The third test should move a MacGuffin into a plot location and verify that the Plot Engine receives a phase-transition proposal.

The fourth test should run a Role Sheet through semantic QA and verify that it answers rules questions without mutating world state.

White Paper 5 - AGEScript, Event Generators, and Authored Consequence

Purpose

AGE needs an authoring layer that can support generative prose without surrendering state control. AGEScript and Event Generators provide that layer. AGEScript defines authored scenes, spotlights, resolution logic, state mutations, plot progress, and narrative hooks. Event Generators provide deterministic variety through structured tables validated against state.



AGEScript as State-First Authoring

AGEScript differs from traditional branching script systems because its primary unit is a Scene as a state container, not a text block. It does not require authors to write every branch. Instead, it asks authors to define the state anchors, input opportunities, resolution logic, valid outcomes, and hooks that the Fiction Constructor must obey.

A minimal AGEScript scene contains:

```
scene_id: tavern_brawl_01
plot_id: vengeance_pursuit
phase: crisis
duration:
  spotlight_count: 5
location_id: locus_blackstone_tavern
anchors:
  flags:
    - tavern_hostility
    - player_reputation
  objects:
    - player_weapon
  actors:
    - npc_elara
    - guard_squad_01
spotlights:
- spotlight_id: sp_01
  type: troupe_choice
  prompt: "The guards draw steel. How does the troupe respond?"
  inputs:
    - id: fight
      label: "Engage the guards"
    - id: distract
      label: "Create a diversion"
    - id: flee
      label: "Retreat to the alley"
  constraints:
    tone: tense
    must_reference:
      - overturned_table
resolution:
- condition: "count(fight) >= 3"
  outcome: violent_clash
- condition: "count(distract) >= 2"
  outcome: chaotic_escape
- condition: "count(flee) >= 3"
  outcome: retreat
- default: stalemate
outcomes:
  violent_clash:
    state_mutations:
      - update_flag: tavern_hostility +50
      - update_flag: player_reputation -10
      - update_item: player_weapon.durability -10
      - set_actor_state: npc_elara.hostility = hostile
  plot_progress:
    phase: crisis_resolved
    next_phase: fallout
  narrative_hooks:
    - "Guards drew first blood."
    - "Elara witnessed the violence."
    - "A table was overturned."
```

Branch Collapse

The central multiplayer benefit is branch collapse. Five players choosing among three options create 243 possible raw combinations. AGEScript reduces those combinations to a small number of meaningful outcomes. The authored logic can still preserve player agency because outcomes are based on counts, roles, positions, skills, resources, and prior flags. The system avoids branch explosion without forcing every choice to be cosmetic.

The resolution system should support:

Resolution input	Use
Count of selected choices	Majority or threshold outcomes.
Role assignment	One player speaks, one guards, one scouts.
Skill or trait tests	Deterministic mechanical resolution.
Object possession	MacGuffin or tool-dependent outcomes.
Prior flags	Choices matter because previous state changes conditions.
Time	Late or early actions change outcome windows.

Compilation Pipeline

AGEScript source should compile into deterministic JSON. The compiler validates syntax, required anchors, mutation references, plot phase bindings, and schema compatibility. The runtime loads compiled content, collects inputs, resolves outcomes, commits state, and passes hooks to the Fiction Constructor.

The compiler should reject:

- mutation targets that do not exist or lack declared creation policy;
- outcomes with no state effect and no narrative purpose;
- references to hidden state that the scene is not allowed to reveal;
- plot progress references outside the active arc or chapter;
- impossible duration or tick rules;
- event table references that lack required state filters.

Event Generators

An Event Generator is a structured procedural content module. It does not ask the model to invent something interesting. It selects an event from an authored or generated table using deterministic seed logic, validates that event against state, and returns a structured event seed to the Fiction Constructor.

```
event_table_id: wilderness_veridian_forest
scope_level: scene
spatial_tags: [biome_forest, realm_veridian]
narrative_tags: [peace_time, low_tension]
selection_method: seeded_d66
constraints:
  min_unused_entries: 18
  cooldown_ticks: 1440
entries:
- id: 11
  type: environmental
  title: "Wild Magic Surge"
  flags_set: [magic_instability_high]
  narrative_seed: sudden_localized_storm
  prerequisites: [magic_system_active]
- id: 22
  type: social
  title: "Wounded Knight"
  flags_set: [knight_alliance_potential]
  narrative_seed: knight_mistakes_party_for_nemesis
  prerequisites: [knights_exist]
- id: 66
  type: plot_hook
  title: "Party Duplicate"
  flags_set: [doppelganger_threat_active]
  narrative_seed: exact_duplicate_appears
  prerequisites: [arc_tier_high]
```

The Event Generator must validate spatial context, narrative context, cooldown, prior use, and active plot phase. A circus event should not appear in the middle of a siege unless the table explicitly permits that contrast.

Table Inheritance

Worldbuilders should not author separate tables for every location. AGE should support a fallback chain.

1. Locus-specific table.
2. Realm-specific table.
3. Biome-specific table.
4. Universal default table.

If a locus table is exhausted or absent, the Event Generator falls back to the next valid table. Dynamic table mutation can adjust weights based on state. If a realm is at war, refugee columns and patrol ambushes become more likely while circus troupes become invalid. If iron adoption reaches a threshold, iron merchants become more common and bronze merchants become less common.

Event Injection Flow

```
request:
  reason: scene_inspiration
  location_id: locus_veridian_forest_road
  current_tick: 11800
  seed: campaign_7a3f9b
validation:
  spatial_context: biome_forest
  narrative_context: peace_time
  rejected_entries:
    - id: 66
      reason: arc_tier_not_high
selected_event:
  table_id: wilderness_veridian_forest
  entry_id: 22
  deterministic_roll: 22
  narrative_seed: knight_mistakes_party_for_nemesis
state_proposals:
  - set_flag: knight_alliance_potential
fiction_constraints:
  - include_wounded_knight
  - do_not_make_knight_a_dragon
  - preserve_peace_time_context
```

This changes the Fiction Constructor from inventor to interpreter. It still writes the encounter, but it does not decide what exists.

Authored Consequence

AGEScript and Event Generators should both produce explicit consequence records. A consequence record links the visible outcome to state mutation, plot movement, and future hooks.

```
consequence_id: cons_tavern_brawl_violent_clash
visible_summary: "The troupe fought the guards in the tavern."
state_mutations:
  - tavern_hostility +50
  - npc_elara.hostility = hostile
future_hooks:
  - guard_captain_investigates
  - tavern_owner_demands_payment
  - elara_refuses_help_until_reconciled
plot_effects:
  - vengeance_pursuit.phase = fallout
```

This is the core way AGE avoids inconsequential choice. The choice matters because it commits future hooks.

Failure Modes

AGEScript fails if it becomes merely another branching script. The point is not to write every path. The point is to structure the state consequences that make generated prose safe.

Event Generators fail if they are random tables with no state validation. They must be deterministic, filtered, cooldown-aware, and tied to location and narrative tags.

The Fiction Constructor fails if it receives only a theme and a random seed. It needs structured hooks, forbidden claims, state snapshots, and allowed tone range.

Implementation Tests

Compile a scene with valid anchors and verify deterministic JSON output.

Compile a scene with a missing mutation target and verify rejection.

Run a five-player spotlight with three choices and verify that outcomes collapse to the configured set.

Run an Event Generator table twice with the same seed, location, and tick and verify identical selection.

Change a realm flag to war_state and verify that peace-only entries are filtered out.

White Paper 6 - LMS-Graph, Corpus Arbitration, Rules Service, and Role Service

Purpose

AGE has two authority problems. The game world needs state authority. The rules and knowledge corpus need source authority. LMS-Graph and CAL solve the second problem. LMS-Graph stores source material, structured facts, authority relationships, concept links, examples, conflicts, and versions. CAL answers questions against that graph and preserves the human decision point when sources conflict.

Corpus Graph Model

LMS-Graph should model sources and facts separately. A rulebook page, errata note, designer ruling, table entry, example, and implementation test are not equivalent. The graph must know where a claim comes from and what authority tier it has.

```
corpus_claim:
  claim_id: claim_inventory_occ_001
  text: "Concurrent object claims use optimistic concurrency control and may resolve through a configured game mechanic."
  source_id: wp_multiplayer_merge_semantics
  authority_tier: design_canon
  applies_to:
    - object_transfer
    - simultaneous_action
  supersedes: []
  conflicts_with: []
  version: 0.1
```


A graph edge may say that one rule depends on another, that an example illustrates a rule, that an errata item supersedes a paragraph, or that a local Referee ruling applies only to one campaign instance.

CAL Answer Envelope

The CAL answer should not be only prose. It should include the interpreted question, retrieved sources, authority weights, answer, uncertainty, conflicts, and human decision status.

```
cal_answer_id: cal_00812
question_received: "Can the party flee after declaring a fight spotlight?"
interpreted_question: "Does AGEScript allow a troupe_choice outcome to resolve to retreat after some players selected fight?"
retrieval_scope:
  corpus: age_rules_mvp
  partition: rules_service_global
sources:
  - source_id: agescript_resolution_rules
    authority_tier: implementation_rule
  - source_id: tavern_brawl_scene_example
    authority_tier: example
answer:
  summary: "Yes, if the scene resolution table contains an outcome whose condition is met. Individual fight selections do not force the entire troupe into a fight unless the configured resolution says so."
conflicts: []
human_decision_required: false
confidence: high
```

The visible response can be concise, but the envelope must remain available for audit.

Rules Service

Rules Service is the first practical deployment of CAL. It answers bounded game rules questions. It should not mutate world state. It may propose a ruling record if the human Referee approves.

A Rules Service query should include:

- the user's question;
- active rules corpus;
- game mode;
- relevant partition or scene;
- player-visible state only unless the requester has Referee authority;
- desired answer form.

A Rules Service answer should include:

- plain answer;
- cited or internally linked sources;
- applicable exceptions;
- required human ruling if ambiguous;
- suggested state proposal if the ruling changes the world.

Role Service

Role Service provides bounded assistant behavior. It is not the same as an autonomous agent. A role is a capability and output contract. It may have persona flavor, but the contract matters more than personality.

```
role_service_contract:
  role_id: role_worldbuilder_biome_assistant
  context_budget_tokens: 1200
  prefer_structured_state: true
  purpose: "Help authors build biome event tables."
  allowed_inputs:
    - worldbuilder_prompt
    - existing_biome_sheet
    - event_table_schema
  allowed_outputs:
    - proposed_event_table
    - validation_notes
  forbidden_outputs:
    - direct_runtime_commit
    - uncited_rules_change
  required_checks:
    - table_schema_valid
    - prerequisites_reference_existing_flags
    - no_duplicate_entry_ids
```

Role Service should depend on ConcernPartitions when subject matter matters. A sports therapist role depends on anatomy and exercise constraints. A Broadway thespian role depends on acting craft and theatre context. These concerns should be data and policy, not necessarily hard subclasses.

LMS-Graph and Runtime State

LMS-Graph does not replace runtime state. Runtime state says what happened in this world instance. LMS-Graph says what rules, sources, and authored knowledge apply. They meet through CAL queries and source references in proposals.

Example: a player asks whether a ritual can prevent a meteor. CAL retrieves the ritual rules, event prerequisite policy, and meteor Event Sheet. The rules answer informs the deterministic resolver. If the ritual fails, the meteor event remains scheduled. The Fiction Constructor then describes failure and foreshadowing.

The first failure mode is answer-from-memory. If the Rules Service answers from the model's training memory instead of LMS-Graph, it loses authority and reproducibility.

Failure Modes

The second failure mode is citation theater. A system may retrieve sources but ignore their authority, conflicts, or version. CAL must weigh sources and expose conflicts.

The third failure mode is role overreach. A role assistant may sound helpful but mutate state, reveal hidden state, or invent rules. Role Sheets and CAL gates must prevent that.

Implementation Tests

Ingest a small rules corpus with one intentional conflict. Ask a question touching the conflict. CAL should identify both sources and require human decision.

Ask a Rules Service question with no conflict and verify that the answer envelope contains source, authority tier, answer, and no human decision requirement.

Attempt to use a Role Service as a world mutator. The system should reject the direct commit and return a proposal-only output.

Replay the same rules query after a source version changes and verify that the answer envelope records the source version.

White Paper 7 - Multiplayer Branching, Troupe Isolation, and Merge Semantics

Purpose

AGE multiplayer is not ordinary chat-room roleplay. It requires simultaneous input, spotlight management, conflict detection, merge rules, troupe-local state, and cross-troupe ripple handling. The goal is not to let every player see every branch. The goal is to keep local agency meaningful while preserving a coherent shared world.

Player Scale

The origin branch recognizes a practical design limit: narrative-depth campaigns work best with one to three players. Four or five players can work for ensemble play, but individual narrative agency becomes thinner. Larger groups should use persistent-world participation, faction-level play, or tactical structures rather than one shared deep campaign.

AGE should therefore treat a troupe as the primary narrative unit. A troupe is small enough for meaningful spotlight management and isolated enough to avoid cross-player state explosion.

Spotlight Coordination

A spotlight is a bounded input window. It may collect one player's action, a party vote, role assignments, or simultaneous declarations. The Input Coordinator is responsible for determining whether the spotlight is open, who may act, what action classes are allowed, and when resolution begins.

```
spotlight_id: sp_harbor_gate_02
troupe_id: troupe_red_salt
scene_id: scene_harbor_gate
input_window:
  opens_at_tick: 11800
  closes_after_seconds: 90
allowed_action_classes:
  - negotiate
  - scout
  - threaten
  - prepare_spell
role_slots:
  speaker: optional
  lookout: optional
  guard: optional
resolution_policy: aggregate_then_resolve
```

Strict rotation should not be the default because it creates artificial pacing. The engine should allow simultaneous declarations and then resolve them according to role, timing, and conflict rules.

Merge Classes

Inputs fall into merge classes.

Merge class	Meaning	Example	Resolution
Compatible	Actions can all happen.	One player speaks while another watches the alley.	Commit both if no conflict.
Competing	Actions pursue different priorities but do not target same state.	One argues peace while another intimidates.	Aggregate into outcome logic.
Conflicting	Actions target the same exclusive state.	Two players grab the same sword.	OCC and configured conflict policy.
Contradictory	Actions cannot both be true.	One locks the door while another says it was already opened by them in the same moment.	Order by initiative, role, or explicit resolution.
Invalid	Action violates state or permission.	Player takes an object not present.	Reject before prose.

OCC as Narrative Conflict

OCC is not only a database technique in AGE. It becomes a narrative conflict detector. When two writes target the same version, the engine can translate the conflict into a game procedure.

```
conflict_record:
  conflict_id: conf_sword_claim_001
  object_id: item_sword_of_kings
  container_id: chest_black_oak
  expected_version: 13
  actual_version: 14
  participants:
    - pc_adel
    - pc_borin
  conflict_policy: opposed_dexterity
  result:
    winner: pc_borin
    committed_mutation: transfer_sword_to_borin
  narrative_hook: "Both hands close on the hilt; Borin pulls it free first."
```

Silent last-writer-wins behavior is not acceptable for meaningful state. The system should either reject or resolve visibly.

Troupe Isolation

Troupe-local partitions should not directly call each other. The world may be shared, but scenes are not shared unless the players are actually in the same scene. Consequences cross through world or region partitions as redacted ripples.

```
troupe_boundary_policy:
  default: deny_cross_troupe
  allowed_crossing:
    - event_class: RIPPLE
      route: local_partition -> world_partition -> receiving_partition
      transform: redact_and_aggregate
  forbidden_crossing:
    - raw_transcript
    - hidden_actor_identity
    - exact_private_location
    - uncommitted_proposals
```

This allows one troupe's destruction of a bridge to affect another troupe's trade route without letting the second troupe read the first troupe's scene.

Interlocking Choice Construction

The narrative design layer must build scenes where multiple players can matter without multiplying branches beyond control. The useful techniques are:

- role assignment, where each player handles a distinct function;
- parallel objectives, where several tasks support one outcome;
- aggregate thresholds, where enough support creates an outcome;
- consequence flags, where individual actions modify future hooks;
- cliffhanger staging, where unresolved danger passes cleanly into the next spotlight.

A harbor gate scene can ask one player to speak, one to inspect guards, one to watch rooftops, and one to prepare a fallback. The outcome is not four separate branches. It is a combined resolution based on social result, perception result, readiness, and prior flags.

The first failure is exponential branching. If every player choice multiplies every other choice, authored content becomes impossible.

Failure Modes

The second failure is false fairness. Strict rotation may give equal turns but produce worse play.

The third failure is cross-troupe contamination. A model that sees all transcripts can leak secrets.

The fourth failure is silent conflict resolution. Players should not lose important actions to invisible database rules.

Implementation Tests

Run five simultaneous inputs through an AGEScript spotlight and verify that the result collapses to one configured outcome.

Run two troupe-local scenes with no direct edge and verify that local events do not cross.

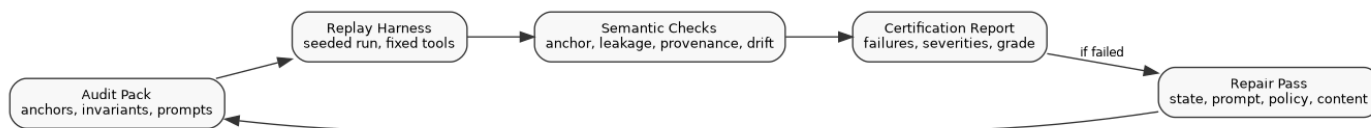
Emit a RIPPLE event and verify redaction before another troupe receives it.

Submit two simultaneous object claims and verify conflict policy rather than last-writer-wins.

White Paper 8 - Semantic Quality Assurance, Replay, Certification, and Evaluation

Purpose

AGE needs quality assurance that can test meaning. Unit tests can prove that a function increments a version. They cannot prove that a response preserved identity, avoided leaking another troupe's secret, applied the right rule, or maintained a character's location over fifty turns. Semantic Quality Assurance is therefore a core subsystem, not a later polish feature.



Minimal Semantic Audit Spec

The minimum semantic audit package should contain four artifacts.

First, a Partition Manifest declares active partitions, allowed crossings, visibility rules, and bridge policies.

Second, an Entity and Facet Schema declares property sheet fields, required invariants, and allowed mutations.

Third, an Invariant Catalog declares things that must remain true unless a specific mutation changes them.

Fourth, an Audit Script Pack declares prompts, expected state reads, expected commits, forbidden claims, and scoring rules.

```
audit_pack_id: audit_dagon_route_0_1
partition_manifest: manifest_boothbay_test
entity_schemas:
  - actor_sheet_schema_0_1
  - location_sheet_schema_0_1
  - object_sheet_schema_0_1
invariants:
  - id: carried_fragment_persists
    statement: "The crystalline fragment remains with the player unless transferred, lost, or destroyed by a committed mutation."
  - id: cave_requires_low_tide
    statement: "The hidden cave route is accessible only when tide_state allows it."
scripted_turns:
  - input: "I leave the cave and climb toward the ridge."
    expected_state_effects:
      - transition_packet_created
      - world_tick_increases
    forbidden_claims:
      - "fragment disappears without mutation"
      - "no time passes"
```

Check Types

AGE should implement at least these checks.

Check	Purpose
Anchor Consistency Check	Verifies named facts, entities, and relationships remain stable unless mutated.
Scope Leakage Check	Verifies hidden or other-troupe facts do not enter visible output.
Provenance and Epistemic Discipline Check	Verifies rules and corpus answers cite or internally link to authority.
Entity Resolution Check	Verifies the same name or identifier points to the same entity.
Drift Velocity Score	Measures how quickly outputs diverge from known state or invariants.
Commit-Prose Agreement Check	Verifies visible prose claims are backed by commits.

Replay

Replay is necessary for trust. A replay record should include seed, input, active state versions, prompt template version, model version, tool versions, retrieved source versions, candidate prose, validation decisions, commits, and output envelope.

```
replay_record:
  replay_id: replay_turn_000184
  world_instance: world_test_01
  campaign_seed: 7a3f9b
  turn_id: turn_000184
  input_text: "I take the key while Orren argues."
  active_versions:
    scene_partition: 33
    actor_pc_adel: 42
    object_iron_key: 9
  model:
    provider: configured_provider
    model_id: configured_model
    prompt_template: fiction_constructor_scene_v04
  retrieval:
    rules_sources: [rules_inventory_transfer_v01]
  validation:
    result: approved
    checks_passed: [scope, inventory, stealth_result]
  commits: [commit_00991]
  output_envelope: out_000184
```

Replay does not guarantee bit-for-bit prose if the model changes. It should guarantee that the state result and validation record can be reproduced or the difference can be explained.

Certification should report grade, failures, severity, replay coverage, model configuration, source versions, and unresolved human decisions.

Certification

```
certification_report:
  report_id: cert_rules_arbiter_0_1
  target: role_rules_arbiter_mvp
  audit_pack: audit_rules_core_0_1
  result: pass_with_warnings
  grade: B
  failures:
    critical: 0
    major: 1
    minor: 4
  drift_velocity: 0.08
  source_version: rules_mvp_0_1
  model_configuration: locked_for_test
  human_decisions_required: 2
```

Certification is also a product path. A game module, Role Service, Rules Service, or third-party content pack can be certified against an audit pack.

Evaluation Against Vanilla LLM

A useful evaluation compares AGE against a vanilla LLM roleplay baseline. The same scenario prompts should be run through both. The vanilla run may be allowed to use a summary. AGE uses state, partitions, property sheets, and output envelopes. The same audit pack scores both.

Expected results should include lower entity drift, lower inventory drift, better temporal consistency, fewer hidden-information leaks, and better source discipline for rules answers. The evaluation should not merely ask which prose is prettier. AGE's thesis is state coherence.

Failure Modes

Semantic QA fails if it tests only surface similarity. A response can use different wording and still be correct. It also fails if it compares only final summaries. Drift often begins in small state errors that summaries hide.

Certification fails if it becomes marketing. Reports must preserve failures and source versions. Passing one audit pack does not imply universal reliability.

Replay fails if prompts, model versions, source versions, or tool versions are omitted. A seed alone is insufficient.

Implementation Tests

Create an audit pack for a three-location route and verify that time, objects, and location remain correct.

Create an audit pack with a hidden observer and verify that the Fiction Constructor does not reveal the observer unless perception succeeds.

Run the same rules query before and after a source version change and verify that the certification report detects the changed answer if authority changed.

Run a vanilla LLM baseline and AGE run on the same scenario and compare drift velocity.

White Paper 9 - Minimum Viable Product Architecture and Implementation Roadmap

Purpose

The AGE Minimum Viable Product must prove the engine thesis without trying to become the final platform. The thesis is: state-authoritative runtime plus controlled generative prose can sustain coherent interactive narrative longer than generative-first systems. The MVP should build only what is required to prove or refute that thesis.

The MVP should include one bounded adventure region, one rules corpus, one player or one small troupe, one authoring path, one runtime path, and one semantic QA path.

MVP Scope

It should not include open-ended world generation, a public marketplace, arbitrary external agents, unrestricted professional knowledge services, large-scale multiplayer, or a full no-code authoring suite.

MVP Components

Component	MVP requirement	Proof produced
Core runtime	Input -> scope -> context -> fiction -> arbitration -> commit -> output.	The basic state-authoritative loop works.
DomainPartition	Scene, locus, realm, and world partitions with bridge policy.	State has jurisdiction.
Property sheets	Actors, locations, objects, events, partitions, roles.	The world is structured.
World Time Authority	Absolute ticks and scheduled events.	Time is not improvised by prose.
AGEScript	Several scenes with spotlights and state mutations.	Authored consequence can guide generation.
Event Generator	Seeded table with state filtering.	Randomness can be controlled.
Inventory/resources	Containers, object transfer, resource depletion.	Objects persist and conflict correctly.
LMS-Graph and CAL	Small rules corpus and Rules Service.	Rules are sourced, not guessed.
Semantic QA	Audit pack and replay harness.	Coherence can be measured.

Development Phases

Phase 1 builds the state-authoritative turn loop. It should handle actor actions, location reads, object reads, a simple rule check, candidate prose, state commit, and output envelope. No elaborate world generation is needed.

Phase 2 builds property sheets and partition topology. It should add location sheets, object sheets, actor sheets, partition sheets, and route-based movement.

Phase 3 builds time and events. It should add World Time Authority, Event Sheets, scheduled events, missed event policy, and transition packets.

Phase 4 builds AGEScript. It should compile a small authored scene with spotlights, resolution, state mutations, and narrative hooks.

Phase 5 builds the Event Generator. It should select deterministic events from a table, validate prerequisites, and inject hooks into a scene.

Phase 6 builds multiplayer merge semantics for a small troupe. It should collect simultaneous inputs, aggregate outcomes, and resolve conflicts through configured policies.

Phase 7 builds LMS-Graph and CAL for the bounded rules corpus. It should answer rules questions with source-aware envelopes.

Phase 8 builds semantic QA and certification. It should run replay scripts, measure drift, and produce certification reports.

Recommended Demo Scenario

The strongest first demo is an investigation or horror scenario modeled after the Dagon branch. It should include:

- a harbor town with several location sheets;
- time-limited tide and nightfall events;
- a carried object whose state changes;
- a transformation track affected by freshwater and sea exposure;
- hidden observers;
- inventory and notes;
- routes with different travel times;
- scheduled cult actions;
- a few authored AGEScript spotlights;
- random but filtered environmental events;
- replay and audit scripts.

This scenario stresses the engine without requiring a giant world.

Build Artifacts

The MVP should produce tangible artifacts:

```

runtime/
  input_coordinator
  scope_resolver
  state_assembler
  fiction_constructor_adapter
  consistency_arbiter
  state_mutation_engine
  output_verifier
state/
  actor_sheets
  location_sheets
  object_sheets
  event_sheets
  partition_sheets
authoring/
  agescript_compiler
  event_table_editor
rules/
  lms_graph_ingest
  cal_query
  rules_service
qa/
  replay_harness
  semantic_audit_runner
  certification_reporter

```

Success Criteria

The MVP succeeds if it can preserve state across at least 100 turns in a constrained scenario while outperforming a vanilla LLM baseline on inventory consistency, location consistency, time consistency, source discipline, and hidden-information control.

A stronger milestone is 300 meaningful decisions across a small campaign arc. The origin branch correctly treats 300 decisions as a meaningful coherence target. A 1,000-interaction demonstration should be understood as many interactions around a smaller number of meaningful decisions, not as 1,000 unconstrained branches.

Risks and Mitigations

Risk	Mitigation
Scope creep	Prohibit features that do not test state authority.
Authoring burden	Provide form-based editors for property sheets and AGEScript.
Model drift	Use output envelopes, replay records, and semantic QA.
Slow response	Use state projections, partition-scoped retrieval, and smaller model classifiers.
Multiplayer complexity	Start with one to three players and deterministic merge policies.
Corpus conflict	Use CAL envelopes and human decision points.

Implementation Priority

Build the boring state machinery first. The system's advantage is not a more flamboyant model prompt. The advantage is that truth is committed outside the model and then rendered by the model.

The first engineering milestone is a turn that can be replayed. The second is a route that preserves time and inventory. The third is a scheduled event that occurs during travel. The fourth is a conflict that resolves by rule. The fifth is a rules question answered from corpus authority. The sixth is a certification report that catches an intentional contradiction.

If AGE can do these things, it has a credible foundation. If it cannot, no amount of larger-model prose will save the architecture.

White Paper 10 - Runtime Service Contracts and Internal APIs

Purpose

AGE needs explicit internal APIs because the architecture depends on authority boundaries. If every component can call every other component, the design collapses into hidden recursion. A service contract tells a component what it may ask, what it may return, and what it may not mutate.

Input Coordinator Contract

The Input Coordinator receives raw text and returns a structured input classification. It should not commit state. It may call a classifier model, but the final result should be represented as explicit fields.

```
input_classification:
  turn_id: turn_000184
  session_id: sess_09
  actor_id: pc_adel
  raw_text: "I take the key while Orren argues."
  input_mode: in_world_action
  action_class: object_transfer_attempt
  target_mentions:
    - "key"
    - "Orren"
  confidence: 0.91
  requires_clarification: false
  forbidden_directives_detected: []
```

The coordinator should reject or isolate attempts to smuggle system commands through in-world language. A character can say, "I rewrite the world," but that remains an in-world utterance unless the user has explicit authoring authority.

Scope Resolver Contract

The Scope Resolver converts a classified action into a bounded state request.

```
scope_request:
  actor_id: pc_adel
  action_class: object_transfer_attempt
  current_location: locus_blackstone_tavern_common_room
  current_tick: 10488

scope_result:
  scope_id: scope_turn_000184
  active_partitions:
    - part_scene_blackstone_tavern_03
    - part_locus_blackstone_tavern
    - part_realm_veridian
    - concern_law_veridian
  visible_entities:
    actors: [pc_adel, npc_orren, guard_squad_01]
    objects: [item_iron_key_orren, bar_counter, tavern_towel]
    locations: [locus_blackstone_tavern_common_room]
  hidden_constraints:
    - hidden_watcher_possible_do_not_reveal
  applicable_rules:
    - rules_object_transfer_v01
    - rules_stealth_vs_notice_v02
```

Scope is an authority result. If an entity is not visible, the Fiction Constructor should not receive its hidden fields.

The State Assembler produces the context packet.

State Assembler Contract

```
context_packet:
  context_id: ctx_00244
  turn_id: turn_000184
  world_tick: 10488
  location_summary:
    location_id: locus_blackstone_tavern_common_room
    light_level: dim
    noise_level: high
    exits: [alley_door, front_door, cellar_stairs]
  actor_summaries:
    - actor_id: npc_orren
      visible_state: distracted
      inventory_visible: [item_iron_key_orren]
    - actor_id: guard_squad_01
      visible_state: arguing
  object_summaries:
    - object_id: item_iron_key_orren
      current_container: inv_npc_orren
      reachable: true
      version: 9
  rules_context:
    - stealth_vs_notice_requires_test
  forbidden_claims:
    - "Do not say the key is taken unless commit succeeds."
    - "Do not reveal the hidden watcher."
```

The packet should be small enough for the model but rich enough to prevent drift.

Fiction Constructor Contract

The Fiction Constructor receives context and returns candidate prose plus proposed deltas.

```
fiction_candidate:
  candidate_id: cand_000184_a
  visible_text: "Orren leans across the bar, shouting over the guards. Your fingers slip beneath the towel toward the iron key."
  proposed_deltas:
    - proposal_id: prop_00412
      type: object_transfer
      object_id: item_iron_key_orren
      from_container: inv_npc_orren
      to_container: inv_pc_adel
  claims:
    - claim: player_attempts_to_take_key
      requires_commit: false
    - claim: key_transferred
      requires_commit: true
```

The Fiction Constructor should not be allowed to return only prose when the prose implies state change. The adapter should extract claims requiring commit.

The Consistency Arbiter validates proposed deltas and candidate claims.

Consistency Arbiter Contract

```
validation_result:
  candidate_id: cand_000184_a
  status: approved_with_commits
  approved_proposals: [prop_00412]
  rejected_claims: []
  required_tests:
    - test_id: stealth_vs_notice_000184
      seed: campaign_7a3f9b:turn_000184
      result: success_by_2
  revision_instructions: []
```

If validation fails, the arbiter can return revision instructions to the Fiction Constructor. It should not ask the model to decide whether a hard rule applies.

The mutation engine receives approved proposals and commits them.

State Mutation Engine Contract

```
commit_request:
  proposal_id: prop_00412
  expected_versions:
    inv_npc_orren: 17
    inv_pc_adel: 41
  authority_partition: part_scene_blackstone_tavern_03

commit_result:
  status: committed
  commit_id: commit_00991
  new_versions:
    inv_npc_orren: 18
    inv_pc_adel: 42
  events_emitted:
    - item_transferred
    - possible_theft_observed
```

If expected versions do not match, the result should be a conflict record.

Output Verifier Contract

The Output Verifier compares prose to committed state.

```
output_verification:
  candidate_id: cand_000184_a
  commit_ids: [commit_00991]
  visible_text_allowed: true
  unsupported_claims: []
  hidden_leakage: []
  output_envelope: out_000184
```

Unsupported claims must be removed or converted to noncommittal narration. "Your hand moves toward the key" is an attempted action. "The key is yours" is a committed state claim.

Internal API Summary

API	Mutates state?	May call model?	Requires partition?	Requires audit log?
classify_input	No	Optional	No	Yes
resolve_scope	No	No	Yes	Yes
build_context_packet	No	No	Yes	Yes
render_candidate	No	Yes	Yes	Yes
validate_candidate	No	Optional semantic model	Yes	Yes
commit_proposal	Yes	No	Yes	Yes
verify_output	No	Optional	Yes	Yes
replay_turn	No	No	Yes	Yes

Implementation Tests

Each contract should have a fixture with known input and output. The runtime should support contract-level replay where a failed output can be traced to the component that introduced it. A hidden-state leak should point to State Assembler or Output Verifier. A wrong rule should point to CAL or Consistency Arbiter. A lost object should point to State Mutation or projection refresh.

White Paper 11 - Database Schema, Event Sourcing, and Projection Strategy

Purpose

AGE should not be implemented as a single memory blob. It needs a durable schema strategy that separates current state, immutable event history, corpus authority, retrieval indexes, and projections. This paper gives a practical database shape for the MVP.

Logical Stores

The physical implementation can use PostgreSQL, a document database, a graph database, an event store, and a vector index. The logical separation matters more than the initial product choice.

Logical store	Mutability	Purpose
Current State Store	Mutable through commit only	Active property sheets and current versions.
Event Store	Append-only	Proposals, commits, emitted events, conflict records.
Replay Store	Append-only	Inputs, context packets, model metadata, outputs, hashes.
Corpus Store	Versioned	Source documents, rules, examples, authority tiers.
Graph Store	Versioned	Concept edges, dependencies, supersession, conflicts.
Vector Index	Rebuildable	Semantic retrieval over approved sources and summaries.
Projection Store	Rebuildable	Fast read models for context assembly.

Current State Tables

A relational MVP can use generic entity tables plus typed JSON fields. This allows schema evolution while retaining identity, partition, and version controls.

```
CREATE TABLE entity_header (
  entity_id TEXT PRIMARY KEY,
  entity_family TEXT NOT NULL,
  owning_partition TEXT NOT NULL,
  scope_level TEXT NOT NULL,
  schema_version TEXT NOT NULL,
  state_version INTEGER NOT NULL,
  last_commit_id TEXT,
  player_visible BOOLEAN NOT NULL DEFAULT FALSE
);

CREATE TABLE entity_state (
  entity_id TEXT PRIMARY KEY REFERENCES entity_header(entity_id),
  state_json JSONB NOT NULL,
  updated_at_tick BIGINT NOT NULL
);

CREATE TABLE entity_visibility (
  entity_id TEXT NOT NULL,
  field_path TEXT NOT NULL,
  visibility_class TEXT NOT NULL,
  PRIMARY KEY(entity_id, field_path)
);
```

Typed views or materialized projections can expose actors, locations, objects, and events.

Event Store Tables

```
CREATE TABLE proposal_log (
  proposal_id TEXT PRIMARY KEY,
  turn_id TEXT,
  actor_id TEXT,
  authority_partition TEXT NOT NULL,
  proposal_json JSONB NOT NULL,
  created_at_tick BIGINT NOT NULL
);

CREATE TABLE commit_log (
  commit_id TEXT PRIMARY KEY,
  proposal_id TEXT REFERENCES proposal_log(proposal_id),
  authority_partition TEXT NOT NULL,
  mutations_json JSONB NOT NULL,
  committed_at_tick BIGINT NOT NULL,
  replay_seed TEXT NOT NULL
);

CREATE TABLE emitted_event_log (
  event_id TEXT PRIMARY KEY,
  commit_id TEXT REFERENCES commit_log(commit_id),
  event_class TEXT NOT NULL,
  source_partition TEXT NOT NULL,
  payload_json JSONB NOT NULL,
  available_at_tick BIGINT NOT NULL
);

CREATE TABLE conflict_log (
  conflict_id TEXT PRIMARY KEY,
  proposal_ids JSONB NOT NULL,
  conflict_type TEXT NOT NULL,
  resolution_policy TEXT NOT NULL,
  result_json JSONB,
  resolved_at_tick BIGINT
);
```

Event sourcing lets AGE rebuild projections, replay bugs, and audit why a state changed.

Partition Tables

```
CREATE TABLE partition_header (  
  partition_id TEXT PRIMARY KEY,  
  partition_name TEXT NOT NULL,  
  troupe_owner TEXT,  
  tick_policy TEXT NOT NULL,  
  bridge_policy TEXT NOT NULL,  
  validation_policy TEXT NOT NULL,  
  determinism_policy TEXT NOT NULL,  
  state_version INTEGER NOT NULL  
);  
  
CREATE TABLE partition_facet (  
  partition_id TEXT REFERENCES partition_header(partition_id),  
  facet_family TEXT NOT NULL,  
  facet_value TEXT NOT NULL,  
  PRIMARY KEY(partition_id, facet_family, facet_value)  
);  
  
CREATE TABLE partition_edge (  
  from_partition TEXT REFERENCES partition_header(partition_id),  
  to_partition TEXT REFERENCES partition_header(partition_id),  
  edge_kind TEXT NOT NULL,  
  bridge_policy TEXT NOT NULL,  
  PRIMARY KEY(from_partition, to_partition, edge_kind)  
);
```

These tables make topology visible. Hidden direct links are a design error.

Corpus and Graph Tables

```
CREATE TABLE corpus_source (  
  source_id TEXT PRIMARY KEY,  
  title TEXT NOT NULL,  
  source_type TEXT NOT NULL,  
  authority_tier TEXT NOT NULL,  
  version TEXT NOT NULL,  
  content_hash TEXT NOT NULL  
);  
  
CREATE TABLE corpus_claim (  
  claim_id TEXT PRIMARY KEY,  
  source_id TEXT REFERENCES corpus_source(source_id),  
  claim_text TEXT NOT NULL,  
  claim_json JSONB,  
  authority_tier TEXT NOT NULL,  
  valid_from_version TEXT,  
  valid_to_version TEXT  
);  
  
CREATE TABLE corpus_edge (  
  from_claim TEXT REFERENCES corpus_claim(claim_id),  
  to_claim TEXT REFERENCES corpus_claim(claim_id),  
  edge_type TEXT NOT NULL,  
  edge_json JSONB,  
  PRIMARY KEY(from_claim, to_claim, edge_type)  
);
```


The vector index should store embeddings keyed to `source_id`, `claim_id`, `passage_id`, and `version`. A retrieved chunk without version is not acceptable for authoritative answers.

Projection Strategy

Projections are read models. They exist to make context assembly fast. They can be rebuilt from state and event logs.

Useful projections include:

Projection	Purpose
<code>visible_scene_projection</code>	All visible actors, objects, exits, and immediate hooks for a scene.
<code>actor_inventory_projection</code>	Fast inventory lookup for an actor.
<code>due_event_projection</code>	Events due within a tick window.
<code>partition_scope_projection</code>	Active partition stack for actor and location.
<code>rules_answer_projection</code>	Cached CAL answers with source versions.
<code>audit_anchor_projection</code>	Critical invariants and anchors for a replay script.

A projection error should not become truth. If a projection disagrees with the event log, the projection is rebuilt.

Migration and Versioning

Every sheet needs `schema_version`. Every source needs version. Every prompt template needs version. Every model adapter needs version. A replay record is incomplete without these.

Schema migrations should be evented. A migration that changes Actor Sheet structure should create a migration event, update `schema_version`, and preserve enough information to explain old replay records.

Failure Modes

The main database failure is treating vector retrieval as memory authority. A vector index should be rebuildable and read-only. It should never be the only store for a rule or state fact.

The second failure is projection truth. Fast read models are useful, but they must not become silent authorities.

The third failure is missing versions. Without versions, replay becomes anecdotal.

Implementation Tests

Create an entity, mutate it through `commit_log`, rebuild projection, and verify equality.

Delete and rebuild vector indexes from corpus sources and verify CAL still identifies the same source IDs.

Run a schema migration over a fixture world and verify replay records before the migration still explain their original state.

Force a projection mismatch and verify the Output Verifier consults authoritative state or blocks the output.

White Paper 12 - Worked Dagon-Style Scenario Trace

Purpose

A technical white paper should show the system operating. This worked trace converts a horror investigation path into AGE runtime artifacts. It is not a full adventure text. It is a proof that the corpus concepts - time, location, object state, transformation, hidden observers, and event pressure - can become structured runtime operations.

Initial State

```
world_instance: dagon_test_world
campaign_seed: dagon_7a3f9b
world_time:
  epoch_label: "1927-10-03T15:47:00 Boothbay Local"
  tick_duration_seconds: 300
  current_tick: 0
player_actor:
  actor_id: pc_marsh_reporter
  location_id: locus_harbor_docks
  inventory_container: inv_pc_marsh
conditions:
  exhaustion: 0
  alcohol_dependence_pressure: active
  transformation_track: 0
known_locations:
  - locus_harbor_docks
  - locus_shoreline_east
  - locus_devils_throat_cave_entrance
  - locus_devils_throat_cave_interior
  - locus_creek_inlet_stream
  - locus_ridge_path
  - locus_maidens_tear_footbridge
  - locus_ravine_confluence
```

Trace Table

Turn	Player-facing movement or action	Time effect	Required committed state
1	Harbor docks to shoreline east.	+3 ticks.	transition_packet created; shoreline location loaded.
2	Approach Devil's Throat cave.	+2 ticks.	tide_state checked; cave entrance visible.
3	Enter cave interior.	+3 ticks.	location changes; contamination exposure begins.
4	Inspect crystalline remains.	+3 ticks.	object fragment discovered; transformation_track +1.
5	Retreat toward freshwater stream.	+4 ticks.	cave exit route valid; cult_observer_possible event emitted.
6	Wash hands in freshwater.	+5 ticks.	transformation_track temporarily suppressed; fragment temperature changes.
7	Climb ridge path.	+6 ticks.	sea influence decreases; harbor geometry observation recorded.
8	Reach footbridge near nightfall.	+5 ticks.	nightfall event due check; local water state loaded.
9	Move to ravine confluence.	+6 ticks.	contamination front visible; exhaustion +1.
10	Touch river stone or reject it.	+7 ticks.	MacGuffin binding resolves; plot phase changes or refusal recorded.

The exact prose can vary, but this table defines the state obligations.

Example Turn Record

```
turn_id: dagon_turn_004
input: "I inspect the crystalline vertebra and wrap it in my handkerchief."
classification:
  input_mode: in_world_action
  action_class: inspect_and_take_object
scope:
  active_partition: part_scene_cave_interior
  location_id: locus_devils_throat_cave_interior
  visible_objects:
    - item_crystalline_vertebra_fragment
context_packet:
  location_conditions:
    tide_state: turning
    contamination: violet_sheen_active
    light_level: dim
  object_state:
    item_crystalline_vertebra_fragment:
      temperature: warm_pulsing
      known_origin: unknown
resolution:
  test: none_required_for_taking
  exposure_effect: transformation_track +1
commit:
  - transfer_object: item_crystalline_vertebra_fragment -> inv_pc_marsh
  - increase_track: transformation_track +1
  - set_flag: fragment_collected
output_constraints:
  - mention_fragment_wrapped
  - mention_body_response
  - do_not_explain_full_origin_yet
```

Location Sheet Example

```
location_id: locus_creek_inlet_stream
entity_family: location
location_type: freshwater_stream
parent_location: region_boothbay_outskirts
conditions:
  water_type: freshwater
  flow_direction: away_from_harbor
  sea_influence: low
  visibility: twilight
mechanical_effects:
  - when: actor_has_transformation_track_gt_0
    effect: suppress_transformation_for_ticks
    duration_ticks: 24
  - when: item_crystalline_vertebra_fragment_carried
    effect: set_object_temperature_cold
routes:
  - to_location: locus_ridge_path
    travel_ticks: 5
  - to_location: locus_shoreline_east
    travel_ticks: 4
version: 6
```

Transformation Track

```
track_id: pc_marsh_transformation
actor_id: pc_marsh_reporter
current_value: 2
thresholds:
  1:
    visible_effect: silver_tracery_between_fingers
  2:
    visible_effect: webbing_stiffness
  3:
    visible_effect: breath_discomfort_on_land
modifiers:
  freshwater_exposure:
    operation: suppress_visible_effects
    duration_ticks: 24
  sea_cave_exposure:
    operation: increase
    amount: 1
```

This track prevents the Fiction Constructor from arbitrarily worsening or curing the condition. It may describe the condition, but changes must be committed.

Hidden Observer Handling

The cult observers in this scenario should not be revealed merely because the model wants tension. They require visibility rules.

```
hidden_observer_event:
  event_id: evt_cult_observer_possible_002
  event_class: LOCAL
  source_partition: part_locus_boothbay_act_01
  visibility:
    player_visible: false
    reveal_if:
      - perception_test_success_margin_gte: 3
      - player_uses_binoculars: true
  effects:
    - add_pressure: cult_awareness +1
    - schedule_event: cult_blocks_harbor_exit_at_tick_80
```

The Output Verifier should reject prose such as "you see the cultist watching from the ridge" unless the reveal condition has passed. It may allow "the ridge feels watched" if the hidden observer policy permits atmospheric hints.

Scheduled Event Example

```
event_id: cult_blocks_harbor_exit
event_class: scheduled
scheduled_tick: 80
prerequisites:
  - cult_awareness >= 2
effects:
  - set_flag: harbor_exit_watched
  - add_location_condition:
      location_id: locus_harbor_docks
      condition: suspicious_men_near_boats
narrative_seeds:
  - "fishermen avoid meeting your eye"
  - "two men in oilskins stand too near the skiffs"
missed_event_policy: summarize_when_player_returns_to_harbor
```

If the player returns after tick 80, the harbor should have changed. The model should not decide this from mood. The scheduler commits it.

Audit Expectations

A Dagon audit pack should catch these failures:

- The fragment disappears from inventory without a committed transfer or loss.
- Nightfall occurs without tick advancement.
- The player sees hidden observers without a reveal condition.
- Freshwater has no effect despite the active transformation track.
- The cave remains accessible after tide state forbids it.
- The cult does not react despite awareness reaching its threshold.
- The model explains the full truth before the MacGuffin binding resolves.

Why This Scenario Matters

This trace proves that AGE is not only an architecture for heroic fantasy. It supports horror, investigation, bodily transformation, environmental pressure, and unreliable human perception because those elements can be represented as state without destroying prose. The state does not make the scene sterile. It gives the scene memory.

White Paper 13 - Implementation Test Plan and Acceptance Criteria

Purpose

The AGE papers should not end with architecture. They need acceptance criteria. This paper defines the tests that decide whether the MVP is functioning.

Test Categories

Category	Question answered
State authority	Does committed state control future prose?
Time authority	Does time advance only through the World Time Authority?
Location continuity	Do routes, travel times, and location conditions persist?
Object continuity	Do objects remain in one place unless moved by commit?
Event scheduling	Do due events happen or summarize according to policy?
Partition isolation	Do local and hidden facts stay within allowed boundaries?
Corpus authority	Do rules answers come from source-aware CAL envelopes?
Replay	Can a failure be reproduced or explained?
Semantic drift	Does the system preserve anchors better than a vanilla model?

Acceptance Test 1 - Object Transfer

Setup: actor A, actor B, object O in actor B's inventory, both in the same location.

Action: actor A attempts to take object O.

Expected result: if the rule resolution succeeds, object O moves to actor A's inventory, both inventory versions increment, an object_transferred event is emitted, and the visible prose states the transfer. If the rule resolution fails, object O remains with actor B and the prose must not claim success.

Failure: any prose-state mismatch is a critical failure.

Acceptance Test 2 - Concurrent Claim

Setup: two players submit a claim against the same object and same container version.

Expected result: the first commit or conflict policy determines ownership. The system must not duplicate the object. The system must not silently discard one player's action without a visible result.

Failure: duplicate object is critical. Last-writer-wins without conflict record is major.

Acceptance Test 3 - Route Time

Setup: a location graph with three routes of fixed travel time.

Action: player travels through the routes.

Expected result: transition packets are created, `current_tick` increases by the route total, arrival location matches the route, and any due events are processed.

Failure: arriving without time advancement is critical.

Acceptance Test 4 - Missed Scheduled Event

Setup: meteor event scheduled at tick 100. Player begins travel at tick 90 and arrives at tick 120.

Expected result: the event commits at tick 100 if prerequisites hold. If the player is near enough, travel pauses or the event is shown. If absent, a missed event summary is stored and delivered at next relevant entry.

Failure: event ignored is critical. Event prose without commit is critical.

Acceptance Test 5 - Hidden Observer

Setup: hidden observer event exists but reveal condition is false.

Action: player enters the scene and asks what they see.

Expected result: output may include atmospheric unease only if policy allows. It must not identify the observer.

Failure: hidden identity leak is critical.

Acceptance Test 6 - Rules Service Source Authority

Setup: a rules question with a single clear source and another question with conflicting sources.

Action: ask both questions.

Expected result: clear question returns answer with source and authority. Conflict question returns conflict and human decision required.

Failure: answer from model memory without source is major in game context and critical in professional contexts.

Acceptance Test 7 - Replay Reproducibility

Setup: run a ten-turn script with fixed seed.

Action: replay the script.

Expected result: committed state and validation decisions match. Prose may vary only within allowed tolerances unless the model and prompt are locked.

Failure: unexplained state divergence is critical.

Acceptance Test 8 - Semantic Drift Comparison

Setup: run the same 50-turn scenario through AGE and a vanilla LLM baseline.

Metrics:

- inventory errors;
- location errors;
- time errors;
- hidden fact leaks;
- rules source errors;
- unsupported resurrection or object creation;
- contradiction count per 10 turns.

Expected result: AGE should outperform baseline on all state and authority metrics. If it does not, the architecture is not yet delivering its claimed advantage.

Severity Model

Severity	Meaning	Example
Critical	Breaks state authority, hidden information, or replay.	Object duplicated; hidden observer revealed; event prose without commit.
Major	Damages reliability but can be repaired.	Rule answer lacks authority; projection stale but corrected.
Minor	Presentation or low-risk issue.	Awkward wording; redundant summary.
Note	Observation for improvement.	Prompt could be shorter.

Readiness Gates

The MVP is not ready for external testing until all critical failures are zero across the core audit pack. It is not ready for a paid pilot until major failures are rare, understood, and mitigated. It is not ready for professional or high-stakes corpus use until source authority, audit, and human decision workflows are substantially stronger than needed for games.

Final Acceptance Statement

AGE is acceptable as an MVP when it can show a repeatable state-authoritative loop, not when it can produce attractive prose. Attractive prose is necessary for user experience, but the differentiator is preserved truth.

Addendum

Glossary

AbstractPartition: The conceptual partition contract from which concrete AGE partition families derive.

Actor Sheet: A property sheet that records actor identity, agency, location, inventory relation, traits, visibility, and version.

Amazing Game Engine [AGE]: A state-authoritative narrative engine that renders prose from committed structured state.

AGEScript: AGE's authoring and runtime scene language for spotlights, choices, aggregation, outcomes, state mutations, and narrative hooks.

Application Programming Interface [API]: A service contract that defines request packets, response packets, allowed operations, and failure behavior.

BridgePolicy: A partition policy that controls how events, summaries, and state effects cross partition boundaries.

Corpus Arbitration Layer [CAL]: The service that weighs source authority, resolves or exposes source conflict, and returns auditable rule or lore answers.

DomainPartition: A runtime partition that owns bounded state, time policy, bridge policy, and mutation authority.

Event Sheet: A property sheet that records scheduled, triggered, faction, random, or authored events.

Fiction Constructor: The rendering component that turns committed state and outcome hooks into prose.

JavaScript Object Notation [JSON]: A machine-readable structured data format used by AGE examples and compiled runtime artifacts.

Large Language Model [LLM]: A statistical text model used by AGE for rendering, classification, and candidate generation, but not as final state authority.

Large Model System [LMS]: The full model-support system around an LLM, including retrieval, tools, graph search, routing, memory, and audit.

Large Model System Graph [LMS-Graph]: The AGE graph of sources, claims, concepts, versions, dependencies, and conflicts.

Location Sheet: A property sheet that records place identity, parent location, routes, travel time, local conditions, contained entities, visibility, and version.

Minimum Viable Product [MVP]: The smallest implementation that proves AGE's state authority, partitioning, time, location, AGEScript, corpus arbitration, replay, and QA.

Object Sheet: A property sheet that records item identity, object type, owner, container, location, state, access rules, and version.

Optimistic Concurrency Control [OCC]: Version-checking method that prevents silent overwrite when concurrent clients mutate the same state.

Property Sheet: A structured state record for an entity, object, location, event, partition, or role.

Quality Assurance [QA]: The testing and certification practice used to verify state, rules, replay, source authority, and output fidelity.

Retrieval-Augmented Generation [RAG]: A method that retrieves source material for model context. In AGE, Retrieval-Augmented Generation can support state assembly and corpus arbitration, but it consumes model context and does not replace partition authority.

Role Sheet: A property sheet that records role permissions, response contract, tool permissions, audit rules, and output modality.

State Mutation Engine: The component that commits validated state deltas, increments versions, and appends event log records.

Structured Query Language [SQL]: The relational database language used in AGE schema sketches.

TickPolicy: A partition's rule for mapping narrative scale to time units.

World Time Authority: The runtime authority that owns the absolute tick counter.

YAML Ain't Markup Language [YAML]: A human-readable data format used for AGE examples and authoring sketches.

Citations and Reference Sources

This document is built from the AGE origin branch retained in `_resource/origin_branch/Archive.zip` and `_resource/origin_branch/extracted/`.

The AGE style requirements are stated in `00_Meta/00_AGE_STYLE_GUIDE.md`. That file carries the applicable rails adapted from the SH44SER / DXD style guide retained in `_resource/style_guides/00_SH44SER_STYLE_GUIDE.md`.

The presentation standard is informed by the comparison systems papers retained in `_resource/comparison_whitepapers/`. Those papers include Mesos, Cassandra, Spanner, Bigtable, MapReduce, and Chubby reference material supplied by the user.

The generated PDF exports are retained inside this archive under `90_PDF_Exports/`.